

An Introduction to Learning in Structured Domains & Learning on Graphs

Alessio Micheli

micheli@di.unipi.it

Work in Progress , please do not circulate!



Dipartimento di Informatica
Università di Pisa - Italy



**Computational Intelligence
& Machine Learning Group**

ToC



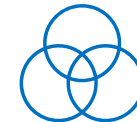
- **Deep Graph Networks**

- With Historical notes



- **Advanced part**

- Is it possible to do DL for graphs **efficiently** (without fully E2E training)?
- The **depth** in DGN is needed, but it can lead to issues: what are the reasons?
- Hence, what is the **interplay** between the depth of models and issues of DGN training?



- **Overview** of some current **advancement in my lab**

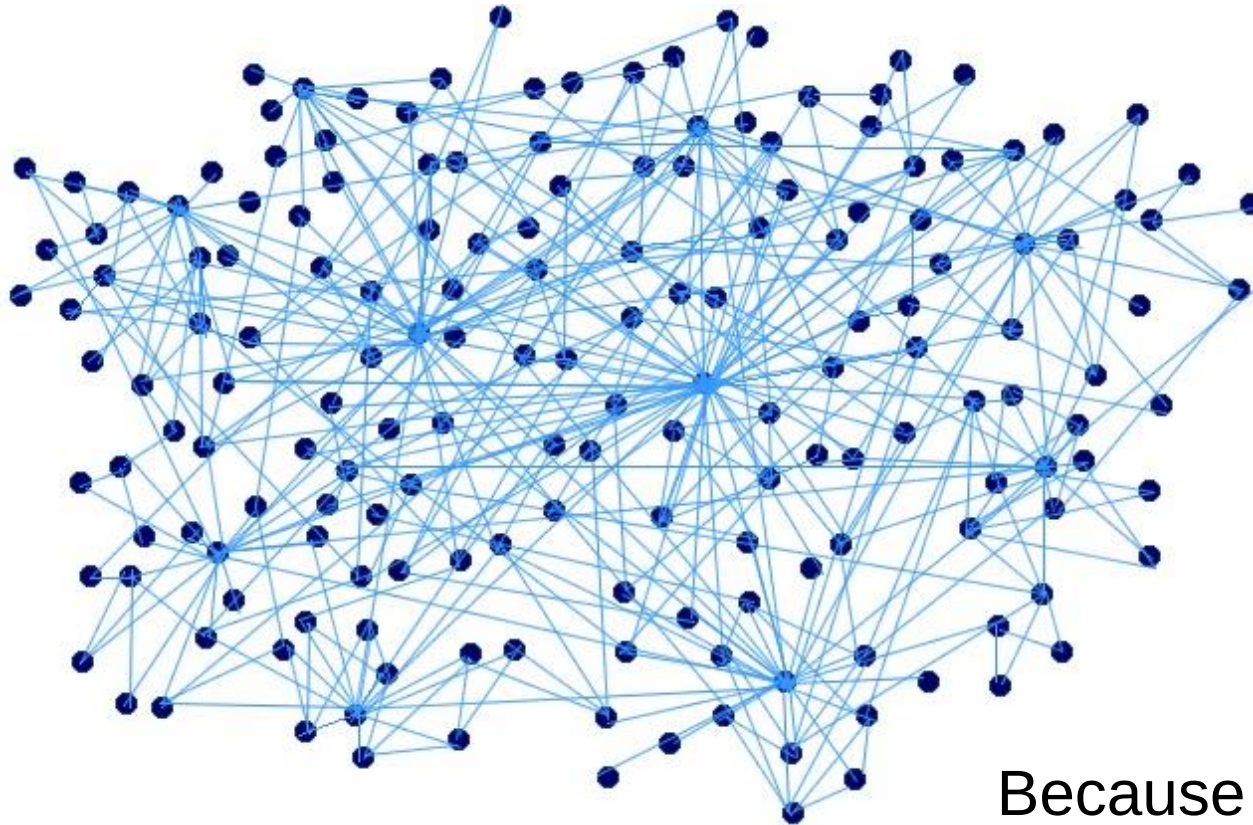
- To open new collaborations



LEARNING IN STRUCTURED DOMAINS

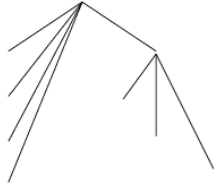
Overview and motivations

Why Structured/Graph Data?

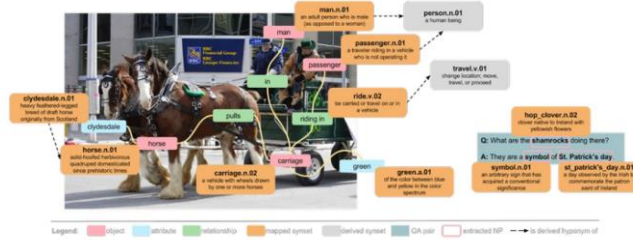


Because data have relationships

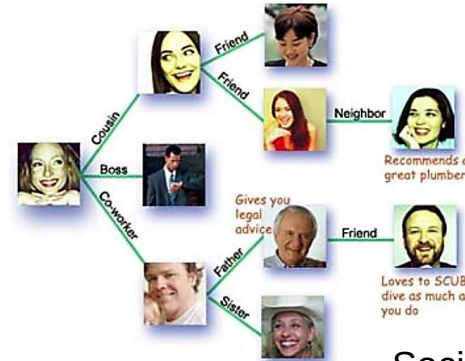
Examples: Graphs & Networks



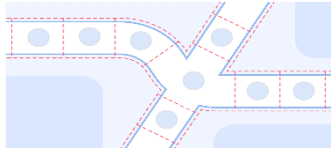
Abstraction of images



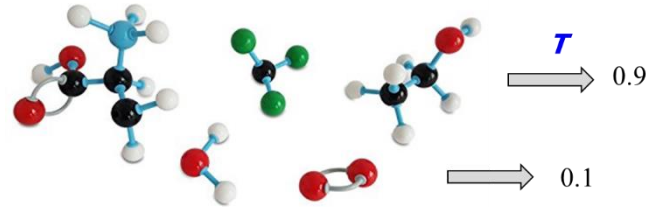
Scene understanding networks



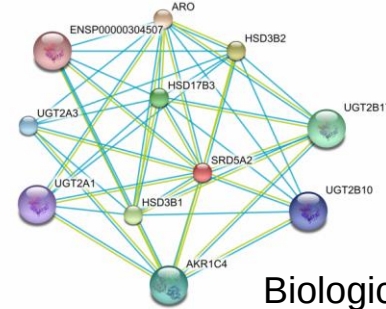
Social networks



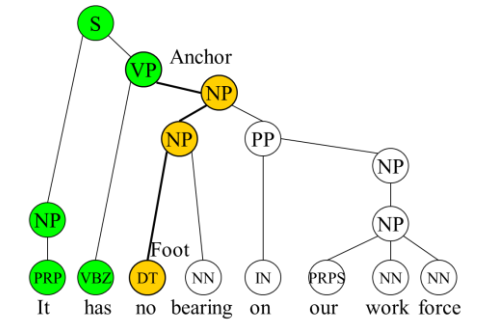
Transportation networks
(traffic predictions on G.maps@DeepMind)



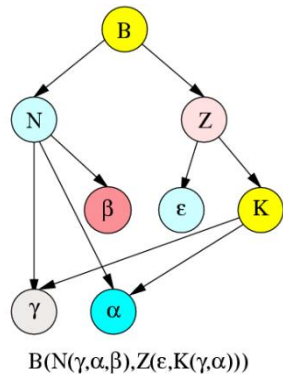
Small molecules (drug design)



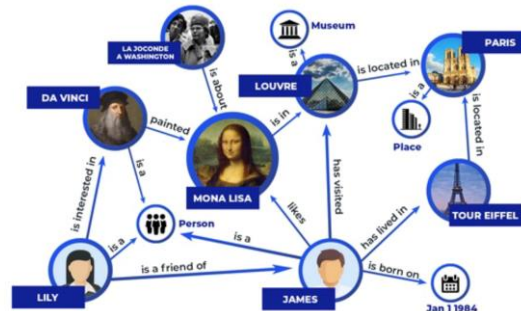
Biological networks
(proteins)



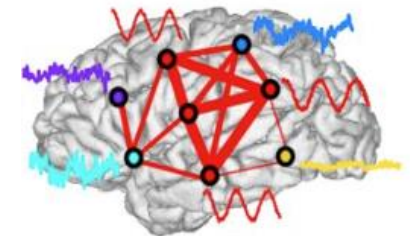
Language parsing



Logic terms

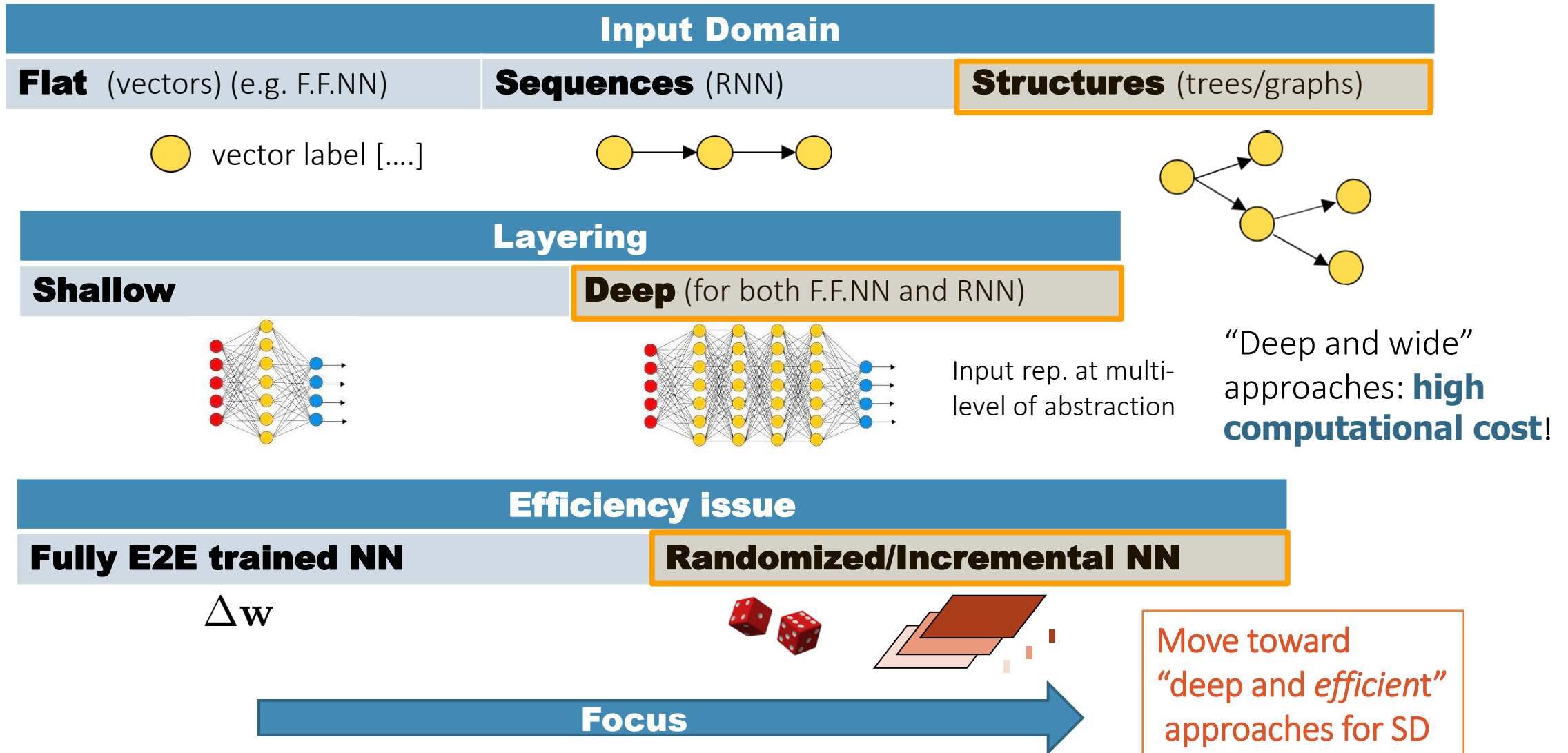


Knowledge graphs



Brain (networks) modeling

The scenario, terms (and trends)

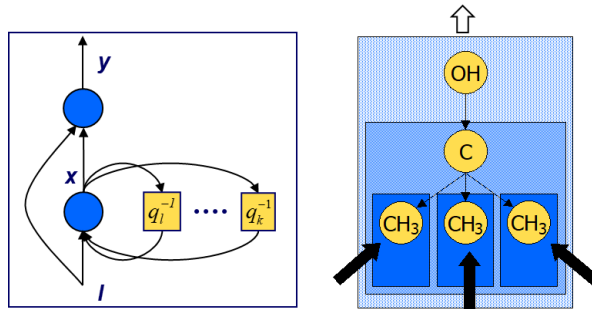


DGN: Origins (the roots, ... and the trees)

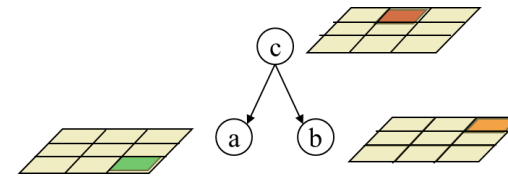


Origins of NNs for **Structured Data**, mainly an Italian history:

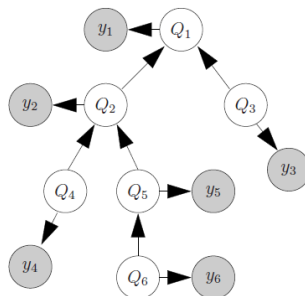
- Started with works on Trees, DPAGs etc. by *Recursive Neural Networks* (A. Sperduti, A. Starita,, P. Frasconi, M. Gori, B. Hammer, A. Micheli, M. Bianchini, F. Scarselli, M. Hagenbuchner, ...) since **end of 90's**, continuing for more than 25 years developing a large **family of models**, e.g.



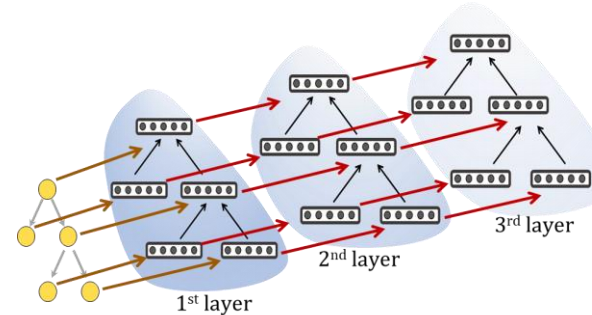
Recursive Neural Networks (1997-2000)



Self-Organizing Map for Structures (2003-2005)



Bottom-up Hidden Tree Markov Models (2012)



Tree ESN and Deep RC for Trees (2010-2018)

Origins of NNs for Structured Data (Trees/DPAGs/...): some references



- A. Sperduti, A. Starita. Supervised Neural Networks for the Classification of Structures, IEEE Transactions on Neural Networks. Vol. 8, n. 3, pp. 714-735, 1997.
- P. Frasconi, M. Gori, and A. Sperduti. **A General Framework for Adaptive Processing of Data Structures**, IEEE Transactions on Neural Networks. Vol. 9, No. 5, pp. 768-786, 1998.
- A.M. Bianucci, A. Micheli, A. Sperduti, A. Starita. **Application of Cascade Correlation Networks for Structures to Chemistry**, Applied Intelligence Journal (Kluwer Academic Publishers), Special Issue on "Neural Networks and Structured Knowledge" Vol. 12 (1/2): 117-146, 2000.
- A. Micheli, D. Sona, A. Sperduti. Contextual Processing of Structured Data by Recursive Cascade Correlation. IEEE Transactions on Neural Networks. Vol. 15, n. 6, Pages 1396- 1410, 2004.
- B. Hammer, A. Micheli, and A. Sperduti.
Universal Approximation Capability of Cascade Correlation for Structures.
Neural Computation. Vol. 17, No. 5, Pages 1109-1159, (C) 2005 MIT press
- B. Hammer, A. Micheli, A. Sperduti, M. Strickert.
Recursive Self-organizing Network Models. Neural Networks, Elsevier Science. Volume 17, Issues 8-9, Pages 1061-1085, 2004.
- C. Gallicchio, A. Micheli. **Tree Echo State Networks**, Neurocomputing, volume 101, pag. 319-337, 2013.
- C. Gallicchio, A. Micheli, **Deep** Reservoir Neural Networks for Trees. *Information Sciences* 480, 174-193, 2019

DGN: Origins



Origins of NNs for SD/**graphs**, mainly an Italian history:

- Moved to **general graphs** (pioneered in [Pisa](#) and [Siena](#), Italy, **2005-2009**):
 1. [Recursive NN: GNN \[1\]](#), GraphESN [3] (see later)
 2. [Layering/Convolutional: NN4Gs \[2\]](#) in constructive version, and then continued as Convolutional NN for graphs or **GCN** etc.
 - A line of research attracting more and more world-wide research since 2015...
- Deep Graph Networks (DGN) framework [4]

[1] F. Scarselli, M. Gori, A.C.Tsoi, M. Hagenbuchner, G. Monfardini. **The graph neural network** model, *IEEE Transactions on Neural Networks*, 20(1): 61–80, **2009**.

[2] A. Micheli. **Neural network for graphs: a contextual constructive approach**, *IEEE Transactions on Neural Networks*, 20 (3): 498-511, **2009**.

[3] C. Gallicchio, A. Micheli. **Graph Echo State Networks**, *IJCNN*, 2010.

[4] D.Bacciu, F. Errica, A. Micheli, M. Podda. A gentle introduction to deep learning for graphs, *Neural Netws.* 2020



IEEE TRANSACTIONS ON NEURAL NETWORKS

A PUBLICATION OF THE IEEE COMPUTATIONAL INTELLIGENCE SOCIETY

www.ieee-cis.org/pubs/tnn



MARCH 2009

VOLUME 20

NUMBER 3

ITNNEP

(ISSN 1045-9227)

REGULAR PAPERS

A New Projection-Based Neural Network for Constrained Variational Inequalities	X.-B. Gao and L.-Z. Liao	373
Analysis of Survival Data Having Time-Dependent Covariates	M. Tsujitani and M. Sakon	389
Kernel-Matching Pursuits With Arbitrary Loss Functions	J. R. Stack, G. J. Dobeck, X. Liao, and L. Carin	395
Multiojective Algebraic Synthesis of Neural Control Systems by Implicit Model Following	S. Ferrari	406
Adaptive Statistic Tracking Control Based on Two-Step Neural Networks With Time Delays	Y. Yi, L. Guo, and H. Wang	420
A Multitask Learning Model for Online Pattern Recognition	S. Ozawa, A. Roy, and D. Roussinov	430
A Self-Organized, Distributed, and Adaptive Rule-Based Induction System	P. Rojanavasu, H. H. Dam, H. A. Abbass, C. Lokan, and O. Pingern	446
Comparison Between Analog and Digital Neural Network Implementations for Range-Finding Applications	L. Gatet, H. Tap-Bételle, and F. Bony	460
An ILC-Based Adaptive Control for General Stochastic Systems With Strictly Decreasing Entropy	P. Afshar, H. Wang, and T. Chai	471
Adaptive Neural Network Tracking Control of MIMO Nonlinear Systems With Unknown Dead Zones and Control Directions	T. Zhang and S. S. Ge	483
Neural Network for Graphs: A Contextual Constructive Approach	A. Micheli	498
Input-State Approach to Boolean Networks	D. Cheng	512

(Contents Continued on Back Cover)



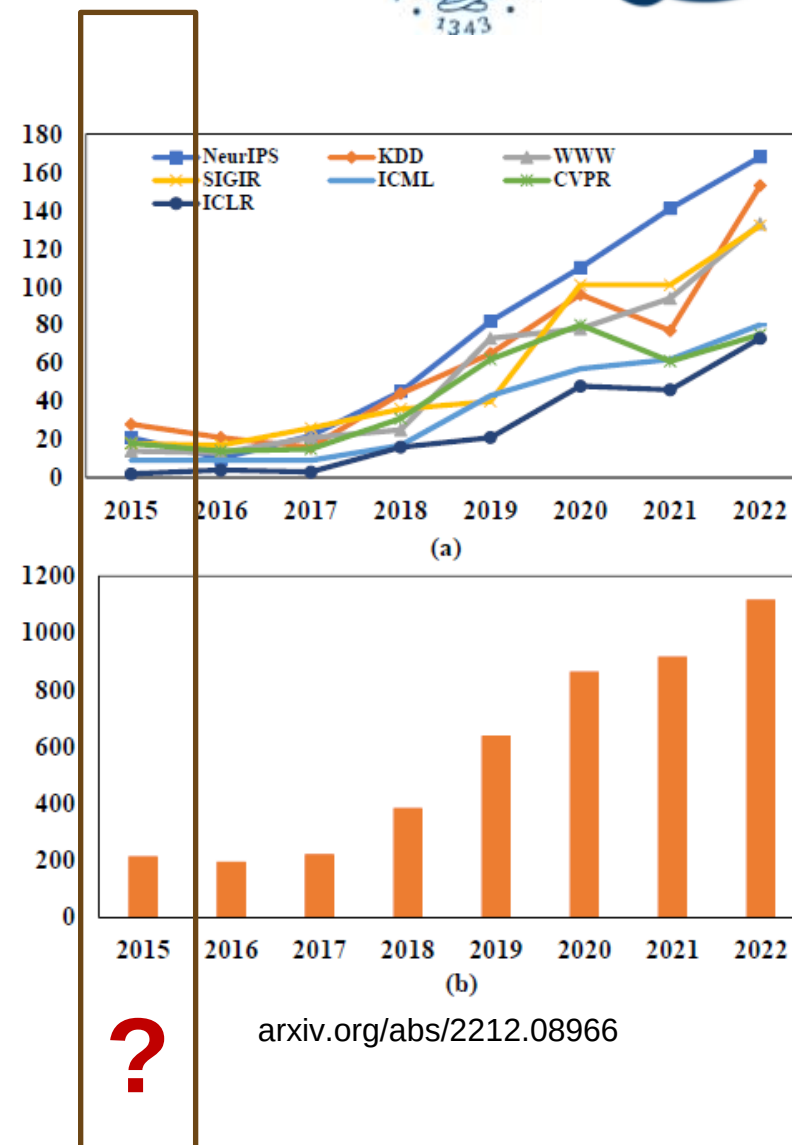
DGN: Evolutions

- Nowadays, a world-wide “exploding” area:
 - Dozens of new research papers in all the top-tier ML venues (literature is rapidly growing)
 - And concrete real-world applications

Number of GL publications in recent years **on top venues**, including NeurIPS, KDD, WWW, SIGIR, ICML, CVPR and ICLR. Figures (a) and (b) depict the number of GL publications at top conferences and the **total number**, respectively

- Including different terminologies and perspectives, e.g.:
 - *Graph Representation Learning*,
 - *Learning on/for/with graphs*,
 - *Deep learning for graphs*
 - **Deep graph networks (DGN) framework** [Neural Netws. 2020]
 - ...

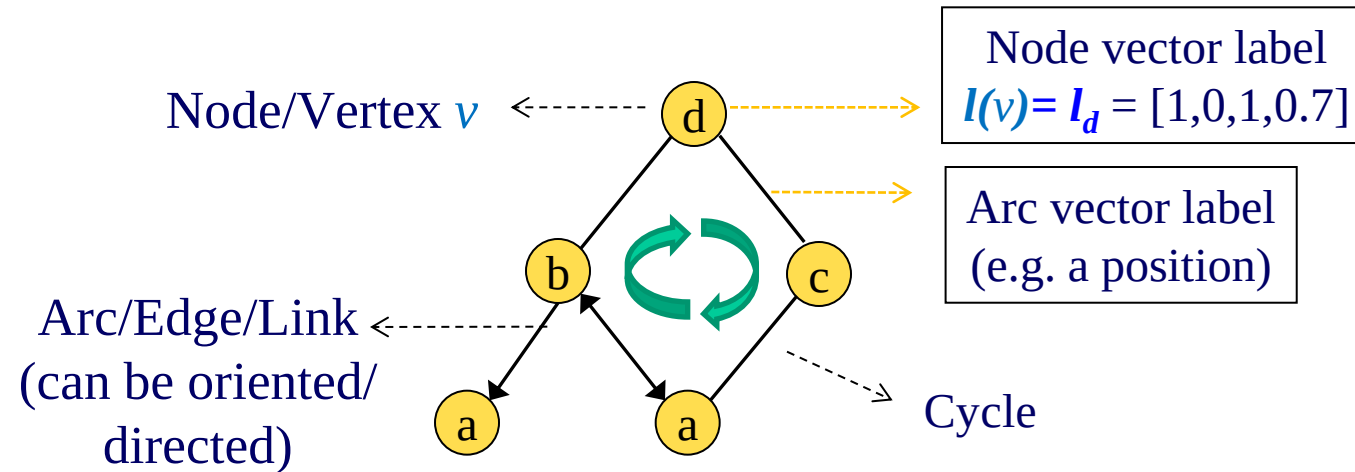
We will see neural models: Neural networks for graphs / Graph Neural Networks (GNN) / Graph Convolutional networks (GCN)/ ...



INTRODUCTION TO DEEP GRAPH NETWORKS

Our graphs (in the following)

Labeled graphs g

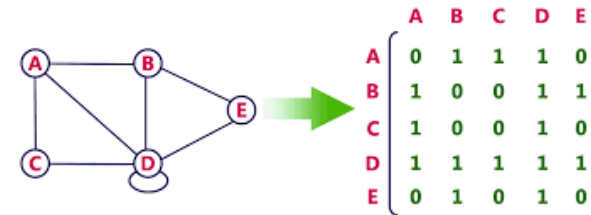


Call $\mathbf{A}$ the adjacency matrix of the graph

Graph Representations

The problem: there has been no systematic way (of general validity for any task) to extract features or metrics relations between examples for SD

- A representation learning instance (extended to SD)!
- **Features based** representations are incomplete (or strongly task-dependent, e.g. topological indexes)
- **Adjacent/incident matrix** representations (or other fixed-sizes representations). Issues:
 - Over-dim./incomplete (wasteful by padding/lose inf.)
 - Alignment among different graphs (!)
 - Topological order (it makes difficult the generalization)
- “The ability to treat the proper ***inherent nature of the input data*** is the key feature for a successful application of the machine learning methodologies”



Learning Models for SD

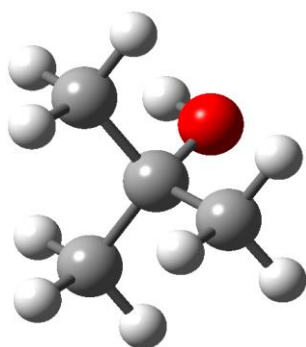
SD + ML = Learning in Structured Domains

Instead of moving *data to models*
(e.g. Graphs into vectors or trees into sequences,
with alignment problems, loose of information, etc.)
we move *models to data*

Example (graphs): Molecules and **Transductions**



- **The problem:** there has been no systematic way to extract features or metrics relations between examples for SD
- **Goal:** to learn a mapping between a structured information domain (SD) and a discrete or continuous space (*transduction*)



$$\text{Property/Activity} = \underset{\textcolor{blue}{T}}{\textcolor{blue}{T}}(\text{Structure})$$



Property Value (regression)
Toxic (yes/no) (classification)

QSPR: Correlate chemical structure of molecules with their properties

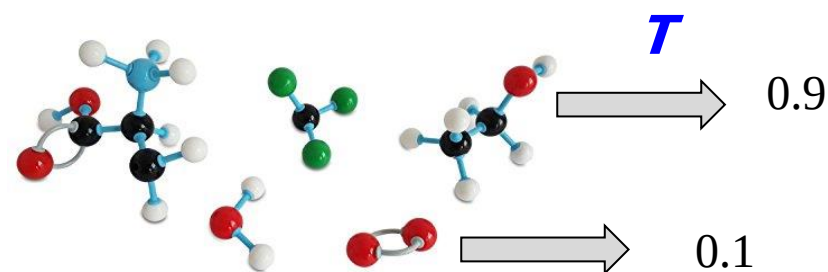
Molecules are not vectors !
Molecules can be more naturally represented by varying size **structures**

Can we predict directly from structures ?

Learning a Transduction

- **Goal:** to learn a mapping between a structured information domain (SD) and a discrete or continuous space (*transduction* T).
- Start with this problem: classify variable size graphs
 - For instance, classify different graphs starting from a training set of known couples as in the molecules example

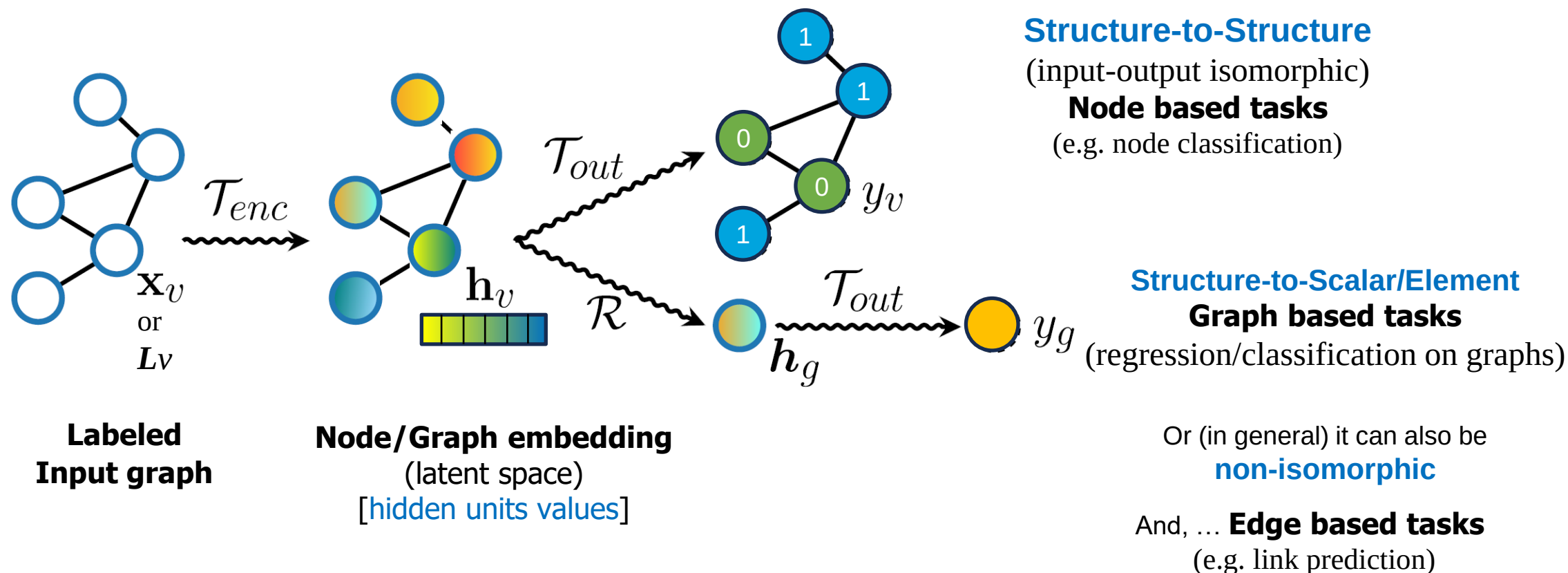
- Given a set of examples ($graph_i, target_i$)
- Learn a hypothesis, i.e. a mapping $T(graph)$



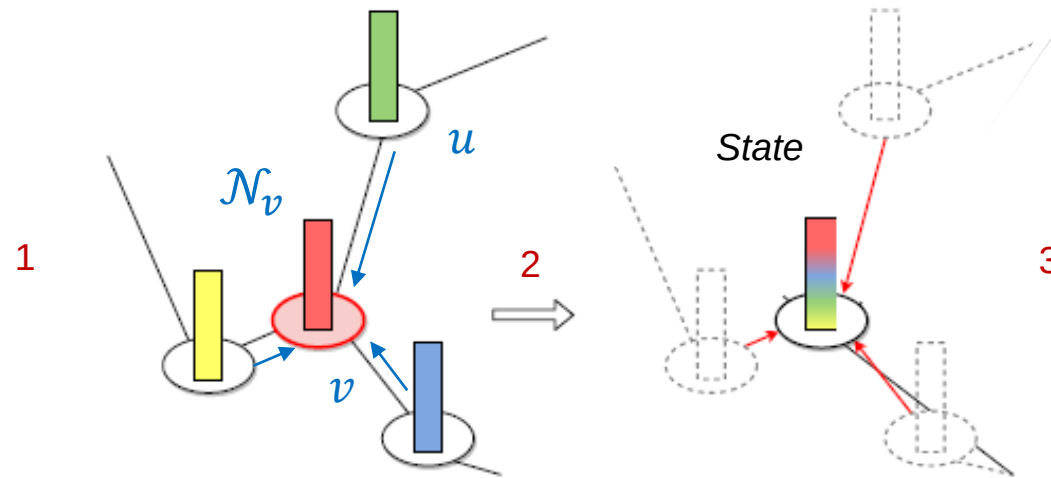
Tasks and Transductions on Graphs



A transduction on graphs can be either



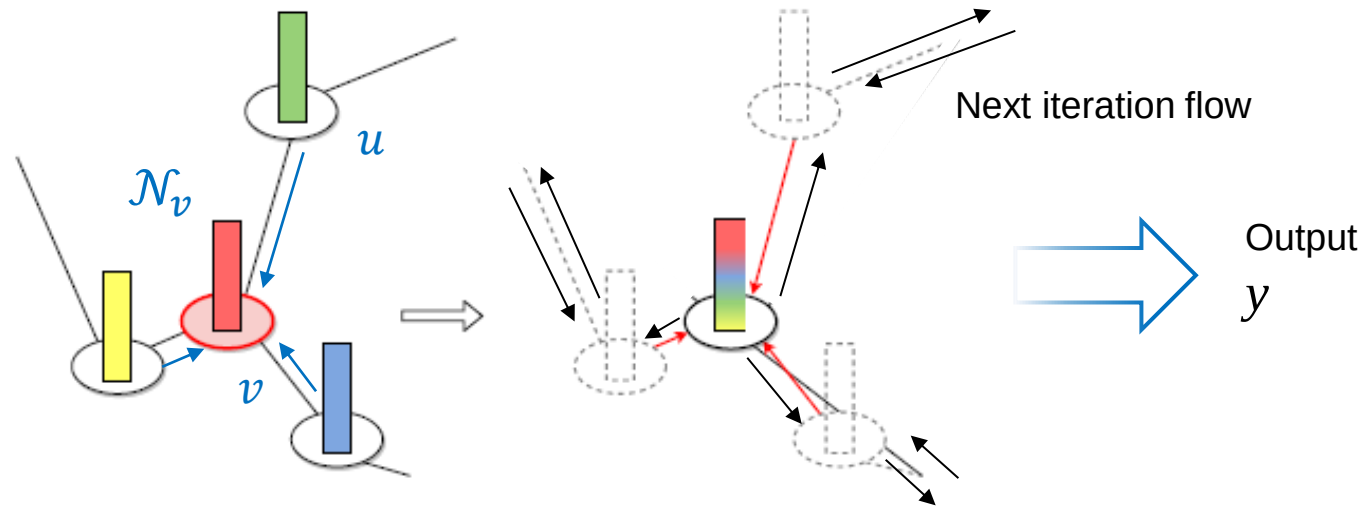
How DGN works? Abstracting «Message passing» concept



Nodes “talk” each other to inform about themselves and to be aware of the surrounding structure, by **messages**

1. Every node in the graph computes a **message** function of the *node label*, the *neighbors* $\mathcal{N}_v$, and the *edges* between them, and send it to its neighbors
2. Nodes **aggregate** the received messages, using a permutation- invariant function (i.e., it doesn’t matter in which order the messages are received).
3. Each node **updates** its attributes (state) combining the messages with free par W

«Message passing» concept (II)

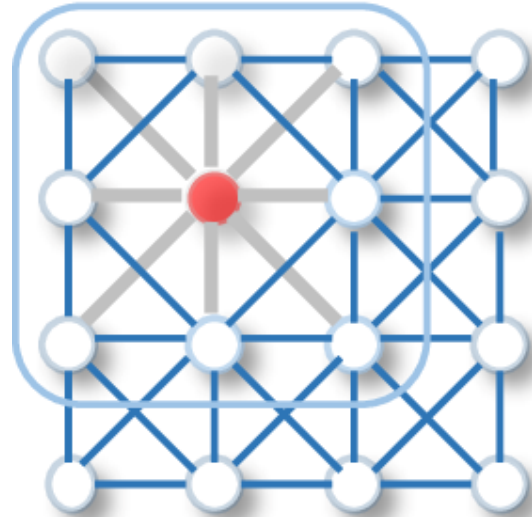


The state does *not* depend only on local *neighborhood* $\mathcal{N}_v$: **Iterations** of this MP mechanism allows the **spread/diffusion of information** for all the nodes to all the others

Finally, an **output** can be emitted **for each node** or **for the whole graph** through a permutation-invariant aggregation of node representations, also called *global pooling* (e.g. a sum or a mean operator)

Layering/ Convolutional approaches:

CNN and CGN both visit each node of the input data, but ...



CNN: Convolutional kernel can be applied on a 2D regular grid (image matrix)

- The CNN takes a weighted average of the pixel values in the neighbor window
- The neighbor of a vertex are ordered and have a fixed size



CNN cannot be immediately applied:

For **graphs** the neighbors of a vertex are *unordered and variable in size*

- We constrain the receptive field to **neighbors**, and we can exploit *weight sharing* to be permutation invariant so to *abstract from any ordering of the nodes*
- **And we extend from local neighbor through layering: see next slides**

Methods: Message Passing

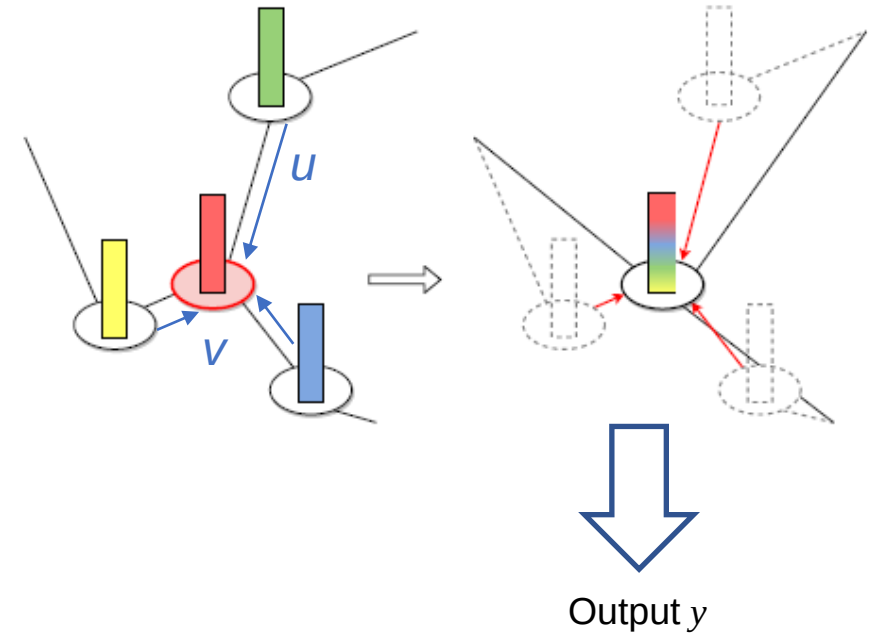


- Iteration (over l) of

Message

$$\mathbf{h}_v^{(l)} = \text{AGG}_{\mathbf{W}^{(l)}}(\mathbf{L}_v, \mathbf{h}_v^{(l-1)}, \{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\}) \quad l = 1, \dots, L$$

- $\text{AGG}_{\mathbf{W}^{(l)}}$ aggregates (propagates) **messages** from the neighbors nodes, it includes a permutation invariant operator over the set of neighbors (e.g. a sum), and combinations of the NN units' activations in $\mathbf{h}$, where $\mathbf{W}$ are the free model parameters. $\mathbf{L}_v$ are the node's labels. $\mathcal{N}$ is according to the adjacency matrix $\mathbf{A}$ of the graph.
- Applying this computation to each node of the graph corresponds to a parallel, unordered **visit** of the graph at each iteration l
- We have a large **flexibility** for implementing these operators, with many model variations:
 - GCN** [Kipf, Welling, ICLR 2017] $\mathbf{h}^{(l)} = \text{relu}(\hat{\mathbf{A}} \mathbf{h}^{(l-1)} \mathbf{W}^{(l)})$, $\hat{\mathbf{A}}$ degree normalized
 - GraphConv** [Morris et al., AAAI 2019] $\mathbf{h}^{(l)} = f(\mathbf{W}_1^{(l)} \mathbf{h}^{(l-1)} + \mathbf{W}_2^{(l)} \mathbf{A} \mathbf{h}^{(l-1)})$
 - GAT** [Veličković et al., ICLR 2018]: Attention-based aggregation



pytorch-geometric

We named as **DGN** framework (including NN, Bayesian, etc) [Neural Netws. 2020]

Methods: Message Passing

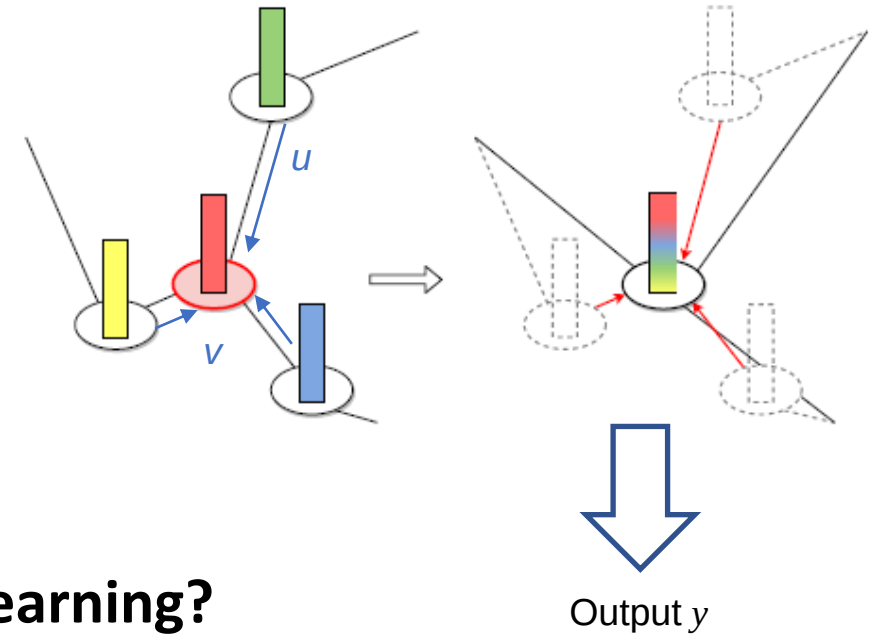


- Iteration (over l) of

Message

$$\mathbf{h}_v^{(l)} = \text{AGG}_{\mathbf{W}^{(l)}}(\mathbf{L}_v, \mathbf{h}_v^{(l-1)}, \{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\}) \quad l = 1, \dots, L$$

- $\text{AGG}_{\mathbf{W}^{(l)}}$ aggregates (propagates) **messages** from the neighbors nodes, it includes a permutation invariant operator over the set of neighbors (e.g. a sum), and combinations of the NN units' activations in $\mathbf{h}$, where $\mathbf{W}$ are the free model parameters. $\mathbf{L}_v$ are the node's labels. $\mathcal{N}$ is according to the adjacency matrix $\mathbf{A}$ of the graph.
- Applying this computation to each node of the graph corresponds to a parallel, unordered visit of the graph at each iteration l



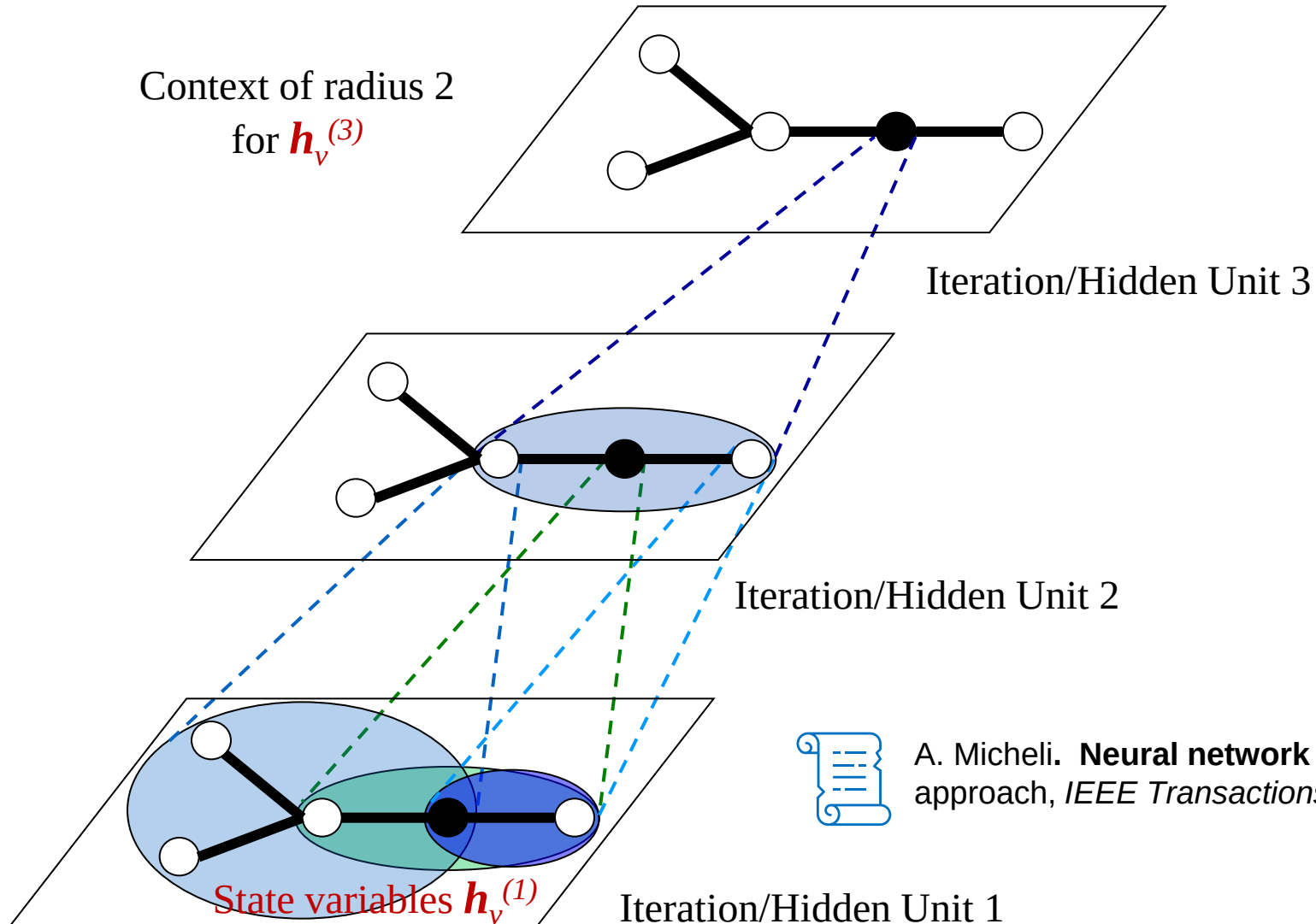
WHY IS IT RELEVANT for Deep Learning?

DGN are *inherently deep*: This is *not* a local process.

While iterating (over layers l), the latent node representations will progressively include the **contextual information** of other nodes beyond the local neighborhood (diffusion).

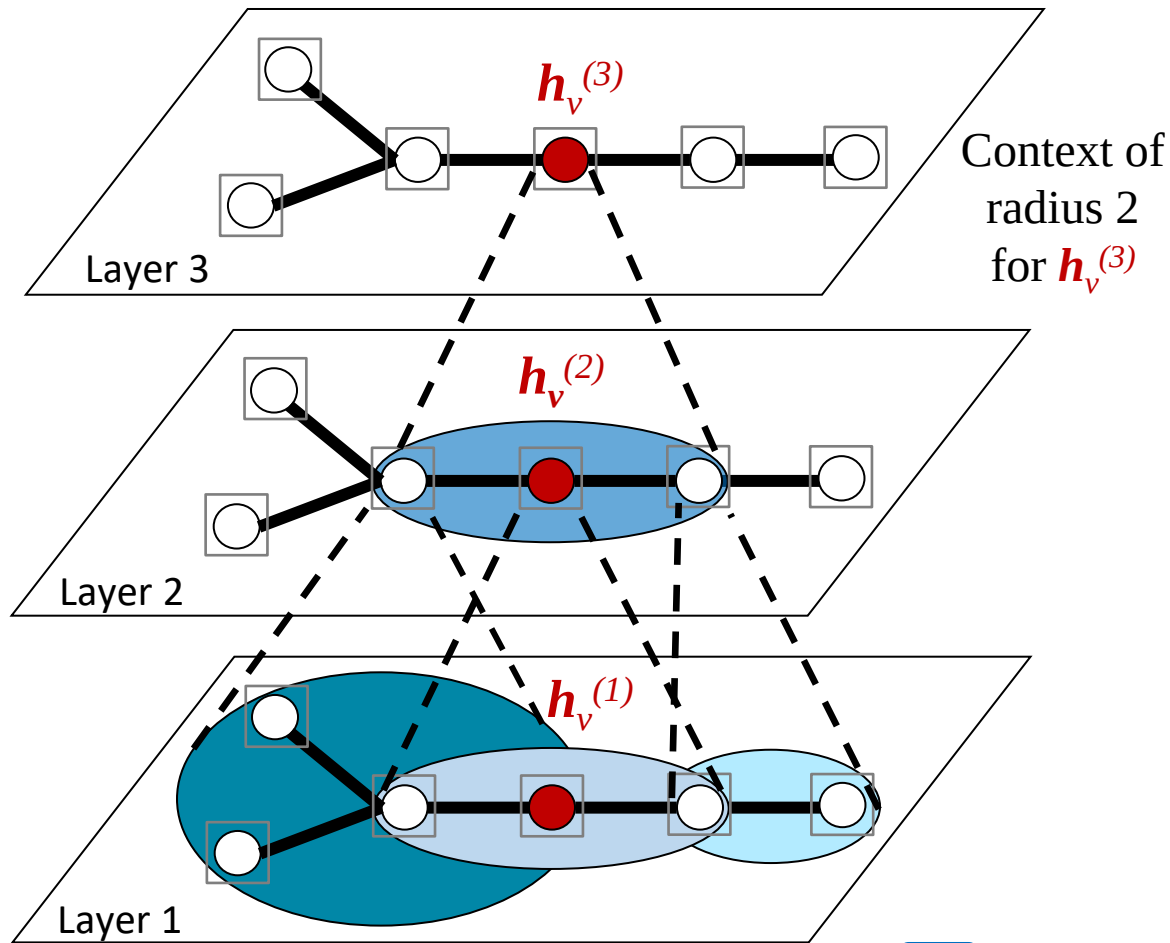
Intuition by the following picture

Evolution of the Context (Compositional) in GCN: the movie



A. Micheli. **Neural network for graphs: a contextual constructive approach**, *IEEE Transactions on Neural Networks*, 20 (3): 498-511, **2009**

GCN & NN4G (2005-2009)



Iterations l of MP are new layers

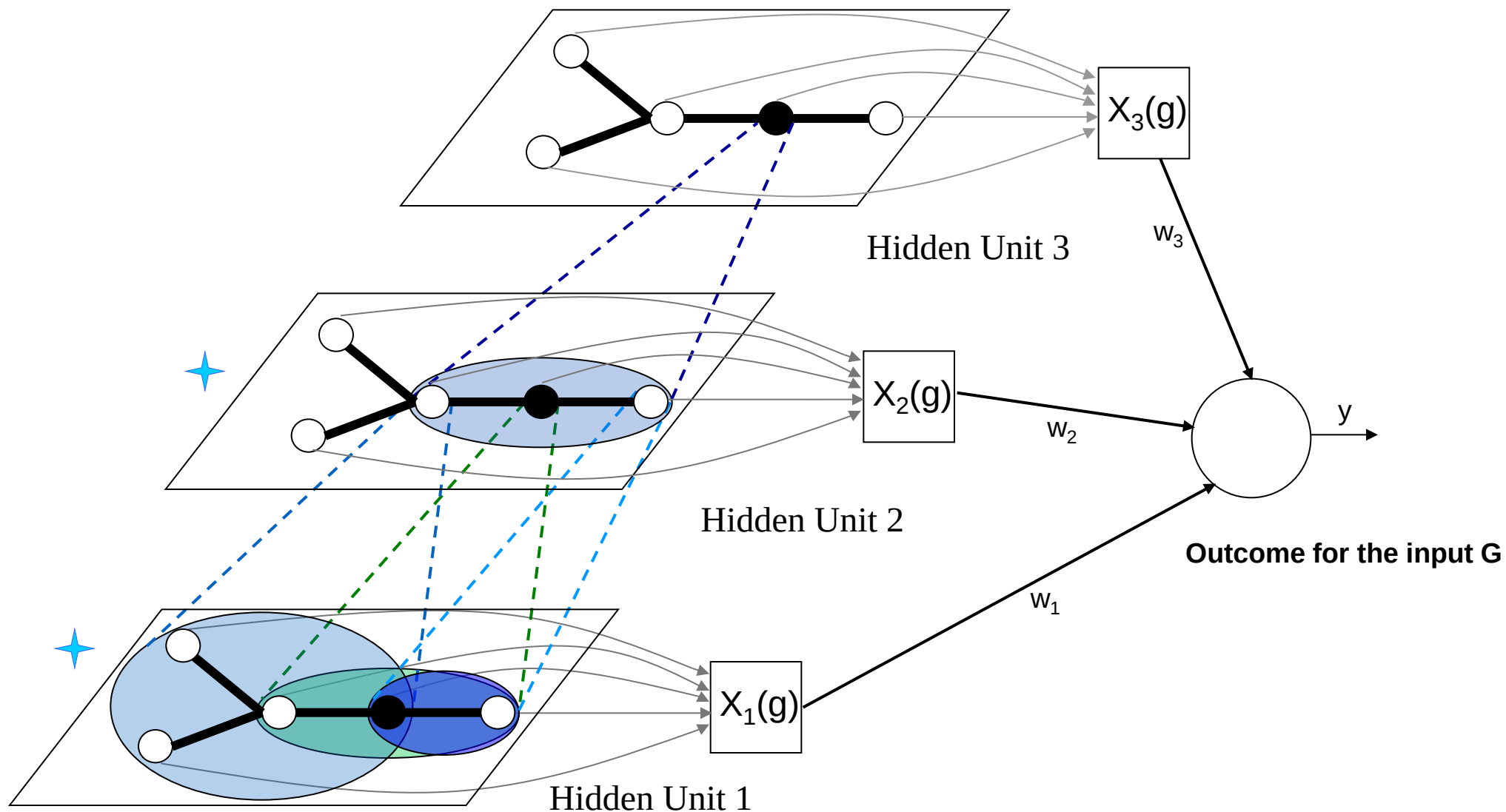
And for each node we compose the context through layers

- The **receptive field** is incrementally extended when the number of layers is increased
- **Depth** is functional to the context growth
- Training the model we learn how to represent the graph:
 - **E2E** training for general **GCNs**
 - **Layer-wise** (incremental) training for **NN4G**
 - (automatic # of layers, no gradient vanish issues, etc)



A. Micheli. **Neural network for graphs**: a contextual constructive approach, *IEEE Transactions on Neural Networks*, 20 (3): 498-511, **2009**

NN4G: The complete model



NN4G: The Equations



Message

$$\mathbf{h}_v^{(l)} = \text{AGG}_{\mathbf{W}^{(l)}}(\mathbf{L}_v, \mathbf{h}_v^{(l-1)}, \{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\}) \quad l = 1, \dots, L$$

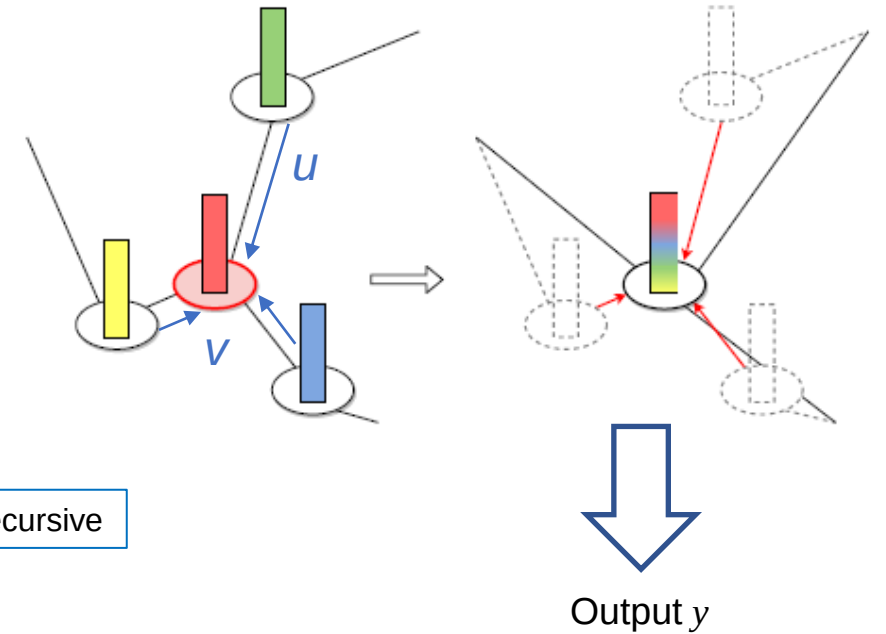
- NN4G computes a (one) state variable for each layer and for each vertex v in this way:

$$\mathbf{h}_v^{(l)} = \begin{cases} h_v^{(1)} = f\left(\sum_{j=0}^{L^v} \bar{w}_{1j} L_j(v)\right) & \text{First layer } (l=1) \\ h_v^{(l)} = f\left(\sum_{j=0}^{L^v} \bar{w}_{lj} L_j(v) + \sum_{j=1}^{l-1} \hat{w}_{lj} \sum_{u \in \mathcal{N}(v)} h_u^{(j)}\right) & l = 2, \dots, N \end{cases}$$

Current Label of v

Context from the Neigh.s
from (all) previous layers

Previous layers: It is not recursive



A. Micheli. **Neural network for graphs**: a contextual constructive approach, *IEEE Transactions on Neural Networks*, 20 (3): 498-511, 2009

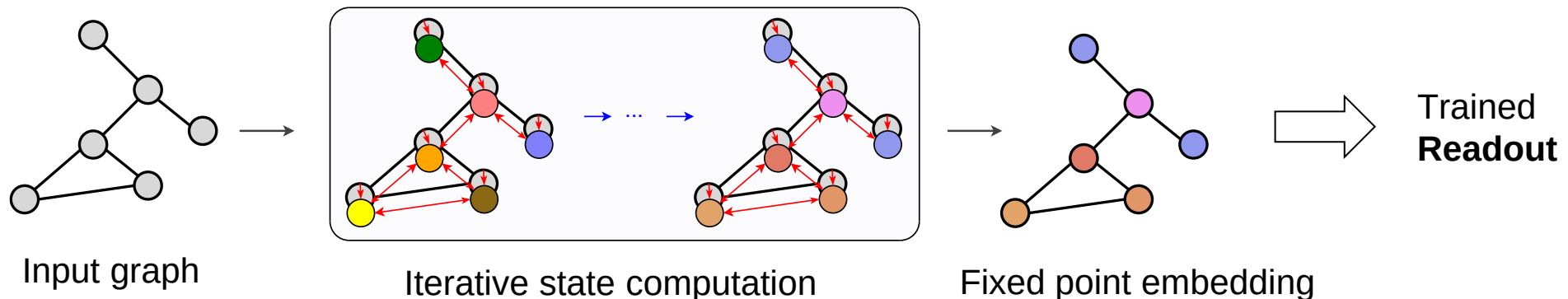
GNN-GESN (2005-2010)



- For the *recursive approaches* (**GNN**, **GESN**) the diffusion process is similar, but l is the number of MP iterations on the same layer (or different layers with shared weights among layers)

$$h_v^{(l)} = \text{AGG}_{W^{(l)}}(\mathbf{L}_v, \mathbf{h}_v^{(l-1)}, \{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\})$$

- Dynamical system**: typically requires convergence to a fixed point as graph embedding, constraining (MP) state dynamics to be contractive
 - GNN** with training for the embedding [Scarselli, et al., TNNLS 2009]
 - GESN** (extending **randomized** Echo State Networks for sequences): **untrained** message-passing function (but direct control of the dynamical system parameters) [Gallicchio, Micheli, IJCNN, 2010; AAAI 2020]



GESN: The Equations



Message

$$\mathbf{h}_v^{(l)} = \text{AGG}_{\mathbf{W}^{(l)}}(\mathbf{L}_v, \mathbf{h}_v^{(l-1)}, \{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\}) \quad l = 1, \dots, L$$

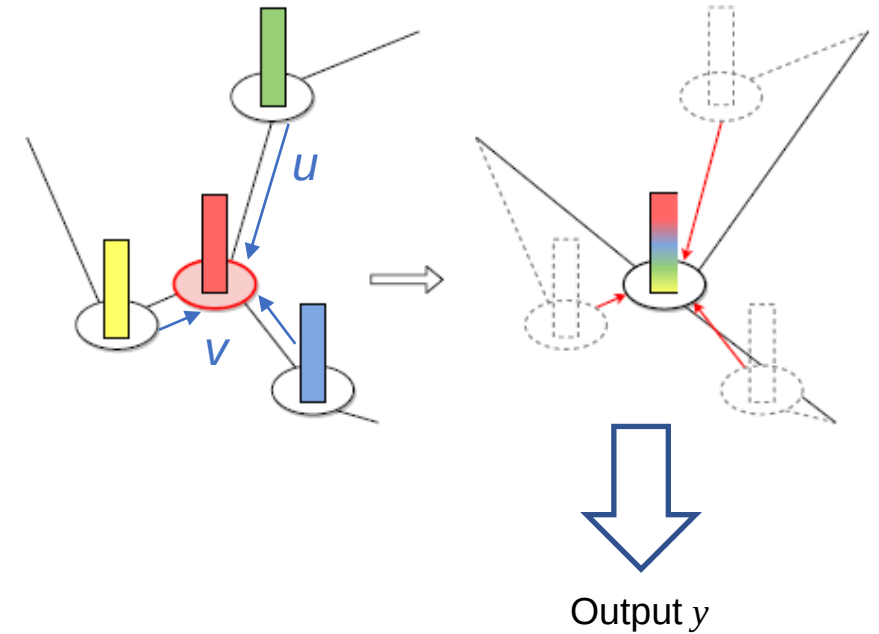
GraphESN compute state variables (here a vector $\mathbf{h}$ of reservoir units) for each vertex v in this way:

GraphESN state transitions (recursive) for l iterations until convergence

$$\mathbf{h}_v^{(l)} = \tanh \left(\sum_{j=0}^{L^v} \bar{w}_{lj} L_j(v) + \sum_{u \in \mathcal{N}(v)} \hat{\mathbf{W}} \mathbf{h}_u^{(l-1)} \right)$$

Connectivity of the Reservoir

Context from the Neigh.s recursively from previous iteration

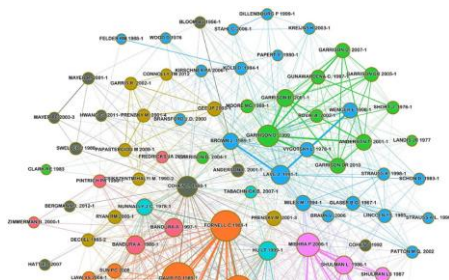


$$\mathbf{y}(\mathbf{g}) = \mathbf{W}_{out} \mathbf{X}(\mathbf{h}(\mathbf{g}))$$

Output for a graph $\mathbf{g}$ by *global pooling* $\mathbf{X}$ and then by a linear **Readout** trained by pseudo-inversion/Ridge Regression

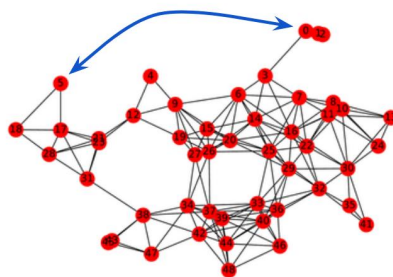
[Gallicchio, Micheli, IJCNN, 2010; AAAI 2020]

Open issues in graph learning models



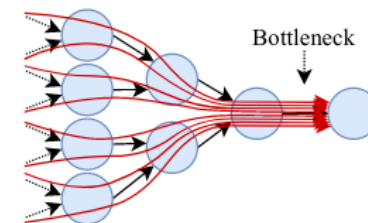
Efficiency

Scaling training to very large graphs, e.g. >100K nodes



Under-reaching

Deep learning models may fail to leverage long range interactions between graph nodes



Expressiveness

Deep learning models may fail to learn meaningful node representations

Our Proposals



EFFICIENCY ANALYSIS

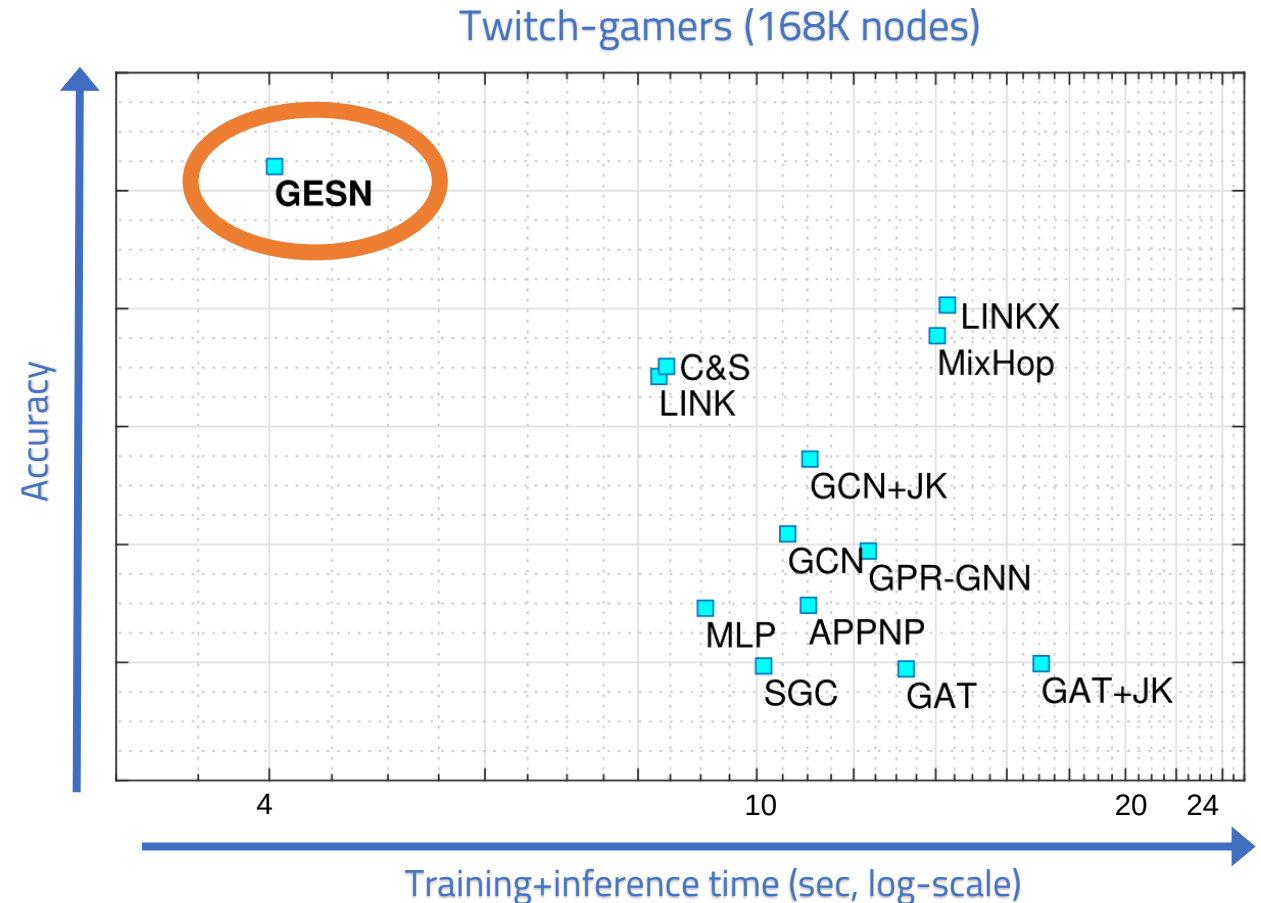
Beyond E2E backpropagation in DGN

GESN for Node Classification (just a glimpse)



- Accuracy equal or better than most end-to-end trained models
- A speed-up upto **10x** compared to other models
- Training scales easily to very large graph, e.g. **>100K nodes**

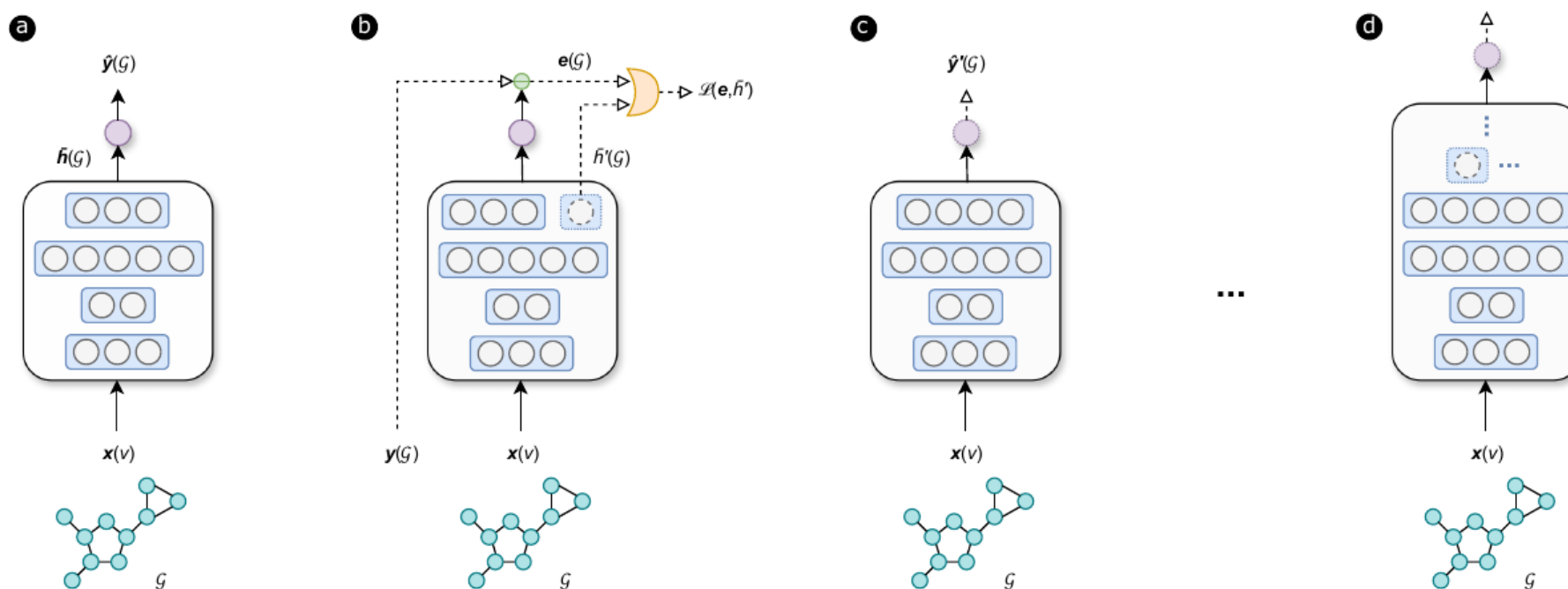
Micheli, Tortorella. *Addressing heterophily with graph echo state networks*. Neurocomputing, 2023



Automatic Architecture design by NN4G+ (New)



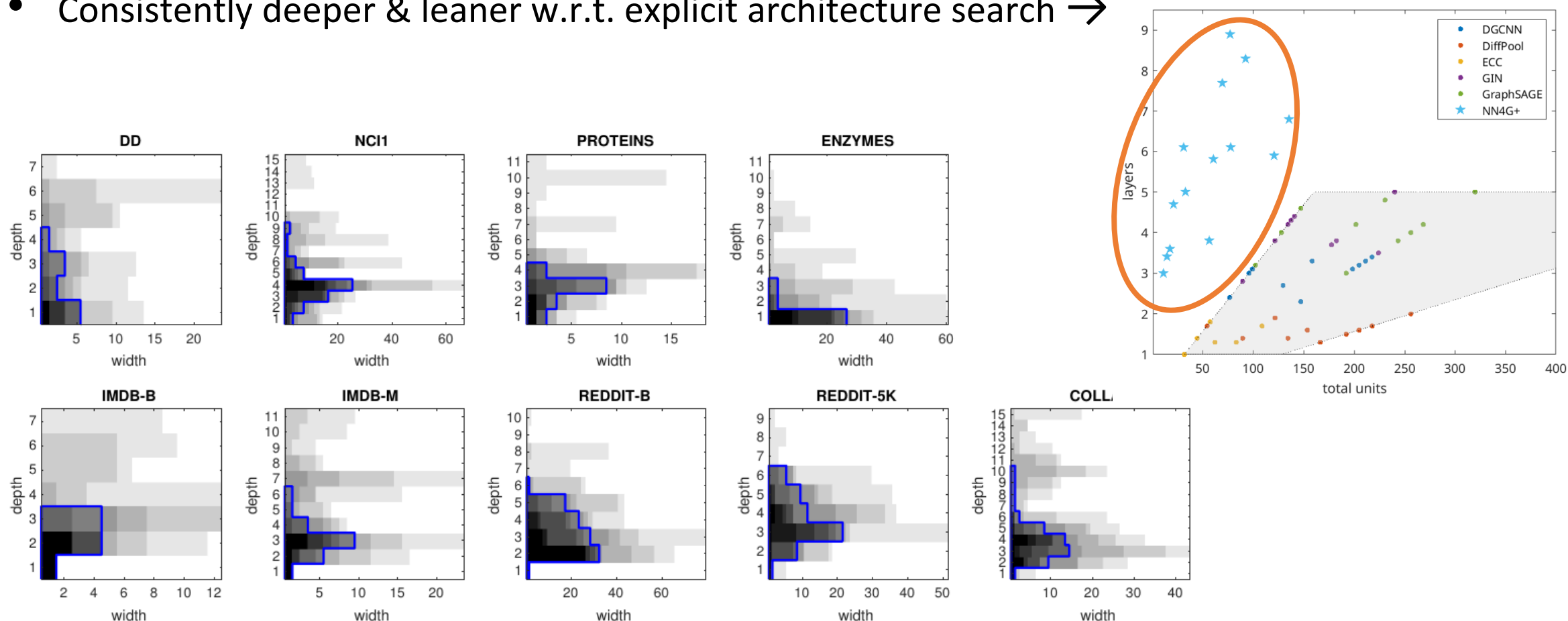
- NN4G+ is built incrementally, with *double synergic expansion* (New!)
 - We train 1 unit at a time: no E2E backprop through layers
 - A layer is expanded greedily until validation accuracy no longer improves
 - New layers are added until validation accuracy no longer improves



NN4g+ architectures are tuned to tasks (New)

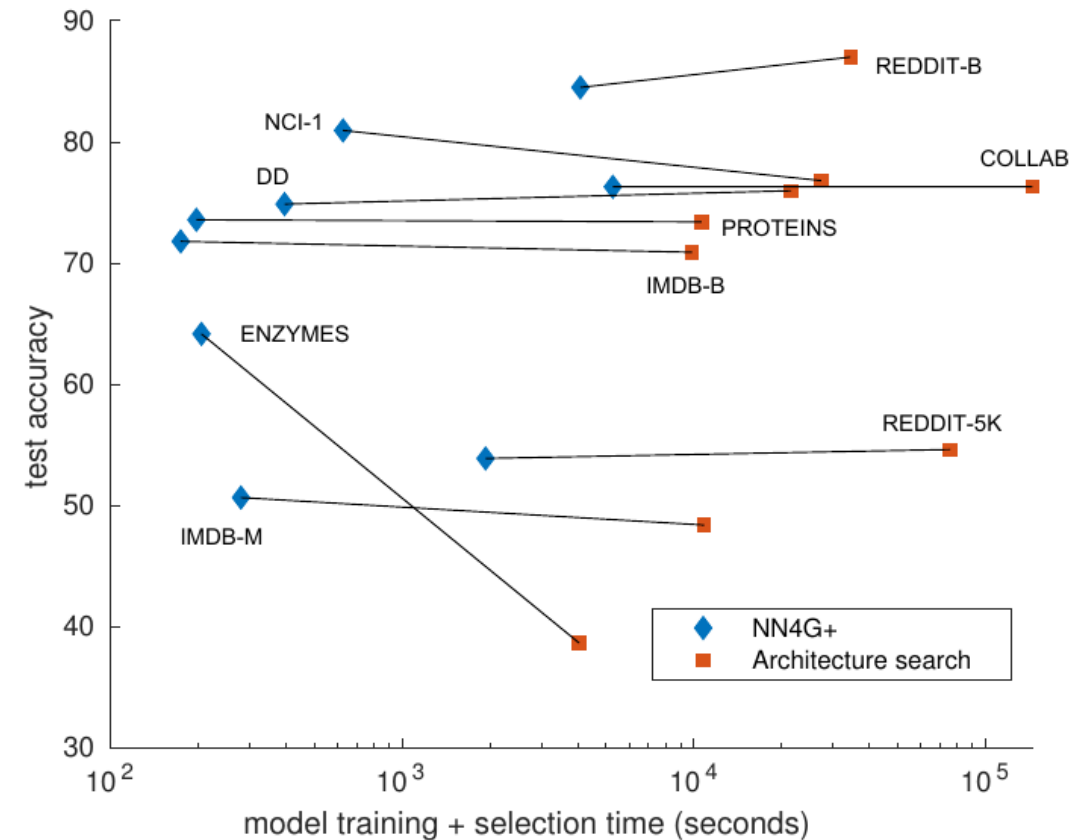


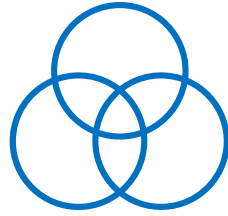
- Consistently deeper & leaner w.r.t. explicit architecture search →



NN4G+ is *Efficient* and Effective (New)

- In general, NN4G+ constructs a neural architecture in *minutes* instead of *hours* wrt end-to-end trained models
- In a fair comparison against *explicit architecture selection*, NN4G+ **speeds up >10×** without penalizing accuracy





ISSUES ANALYSIS

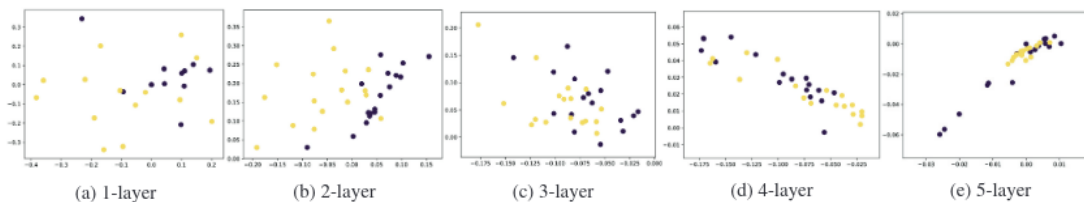
- The **depth** is need but ...
- What are the **issues** in graph representation learning (MP bias)?

MP Issues and Bias: the Over-* cases

- **Over-smoothing**

- Accuracy typically declines beyond 4-5 layers
- Node representations collapse in deeper layers
- Message-passing acts as diffusion
 - *high-level representations of nodes become similar*

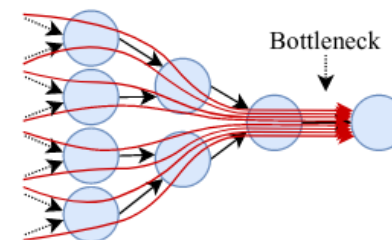
[Li & al. , AAAI 2018]



- **Over-squashing**

- Bottleneck: receptive fields increase exponentially with depth in a fixed-size node embedding dimension [Alon, Yahav, ICLR 21].
- Exponential decrease in **sensitivity** of node repres. to the input features, as the number of layers $h_v^{(L)}$ increases, computed as the Jacobian:

$$\left\| \frac{\partial h_v^{(L)}}{\partial x_u} \right\|$$

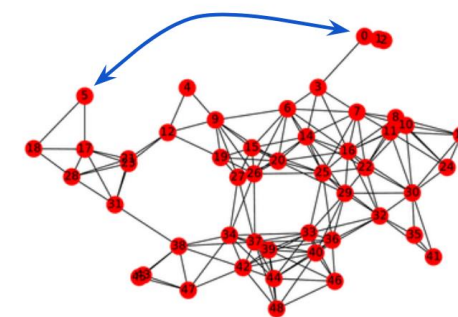


- **For E2E training:** difficulty of backpropagating the gradient through many (MP) layers

- The **interplay** among them is an open research issue

- **Impact 1: Under-reaching**

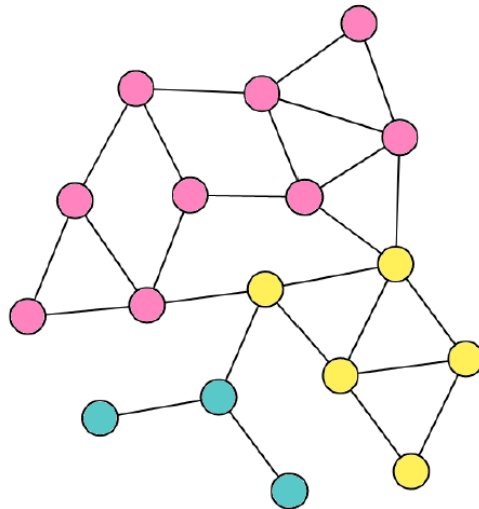
- Over-* prevents the necessary receptive field to learn interactions between distant nodes



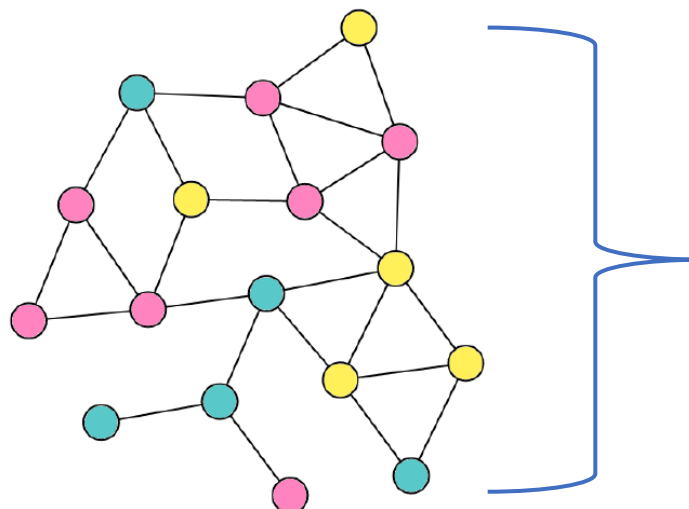
Impact 2: Heterophily Issue in Node Classification



- **Homophilic graphs:** nodes in the same neighborhood mostly share the same class
- **Heterophilic graphs:** nodes belonging to the same class are generally further apart (predictions that rely chiefly on a node's immediate neighbours can be misleading)



high homophily



low homophily

(!)

Potentially **difficult tasks** for standard MP

- impact of the **over-***
- **bias** toward «homogeneous» graphs

A. Micheli, D. Tortorella. *Addressing heterophily in node classification with graph echo state networks*. Neurocomputing (2023).

Some Tools for the Analysis



- (1) **Models** that *decouple* the MP inductive bias of DGNs from E2E training issues (*going beyond backprop for DGN **)
 - **NN4G**: incremental, *avoiding gradient propagation through layers*
 - **GESN**: *no training* for the embedding, direct control of (fixed) spectral norm $\|W\|$, Lipschitz constant of the recursive MP function, and imposing $\|W\| > 1$ to prevent the *sensitivity* w.r.t. the input to be exponentially vanishing (over-squashing)
 - Accuracy comparison
- (2) **Looking inside** the models' layers
 - Spectral analysis (graph signals)

$$\left\| \frac{\partial \mathbf{h}_v^{(L)}}{\partial \mathbf{x}_u} \right\| \leq \left(\prod_{l=1}^L \|W^{(l)}\| \right) \cdot (\mathbf{A}^L)_{u,v}$$

Spectral Analysis: Defs (Tech, Skip)



Let us measure ...

- **Frequency for graphs:** variance of node neighbor signal (label, embedding) over the graph
- **Dirichlet energy (spatial):** $E(\mathbf{h}) = \sum_{v \in \mathcal{V}} \sum_{u \in \mathcal{N}_v} \|\mathbf{h}_v - \mathbf{h}_u\|^2$

Spectral analysis

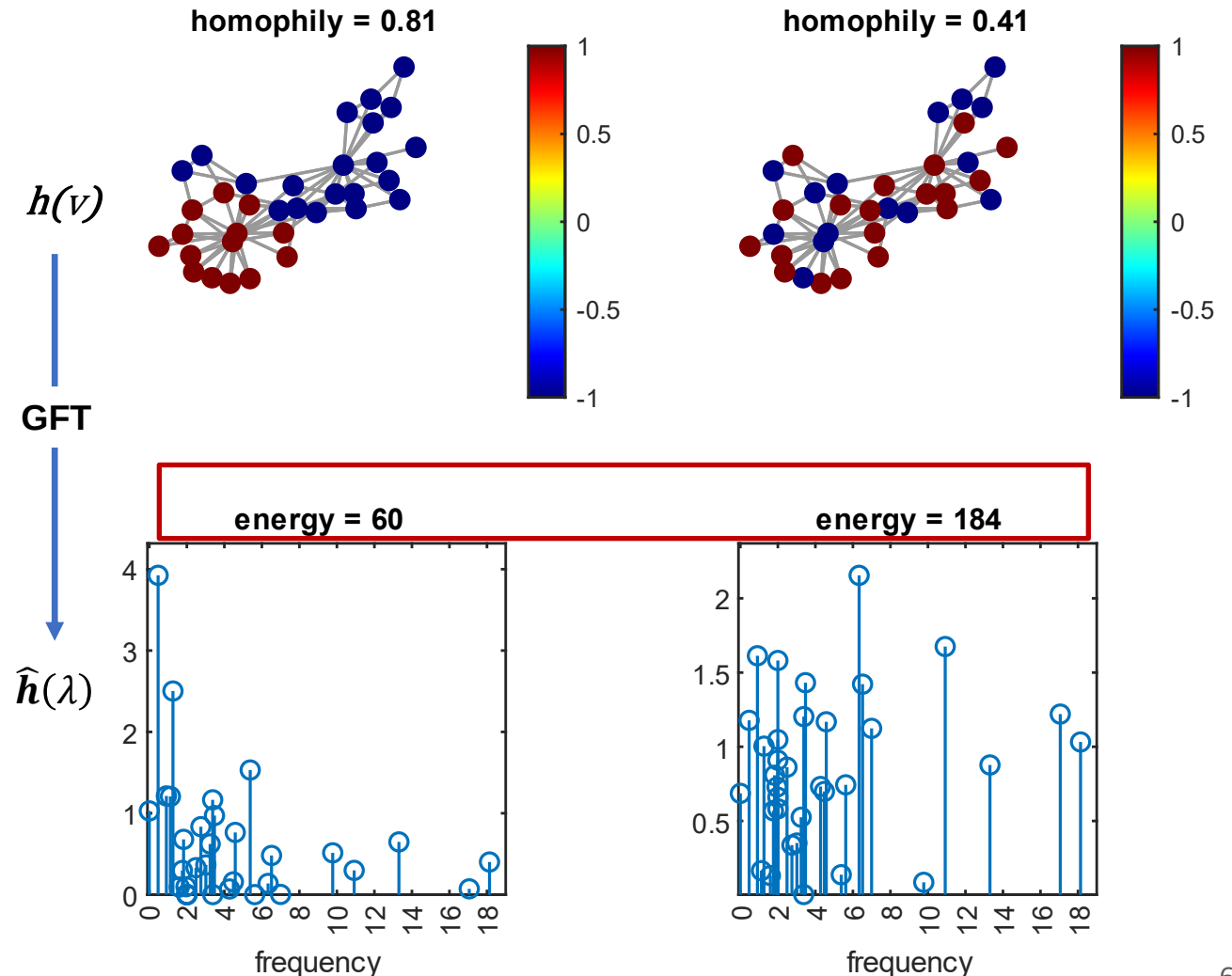
- *Graph Fourier transform* $\hat{\mathbf{h}}(\lambda)$ of the spatial “signal” $\mathbf{h}(v)$, obtained by graph Laplacian ($\mathbf{L}=\mathbf{D}-\mathbf{A}$) eigendecomposition
- The **Dirichlet energy** provides a concise *measure* of how the node feature “mass” $\|\mathbf{h}\|^2$ is *distributed* on the *graph frequency spectrum*, giving an indication of the “smoothness” of the features.
E.g. Low frequency signal $\rightarrow$ low Energy
- $E(\mathbf{h}) = \sum_{i=0}^{N-1} \lambda_i \|\hat{\mathbf{h}}(\lambda_i)\|^2$, where N is the number of nodes, λ_i the eigenvalues (frequencies) and $\|\hat{\mathbf{h}}(\lambda_i)\|$ the magnitude of signal frequencies

Spectral Analysis & Etherophilic Graphs (Tech)



- (High) spatial variability of the graph signal, (high) frequency for graphs, are related to low homophily
 - i.e. **larger spectra by Graph Fourier Transform (GFT) for etherophilic graphs** (on the right)
- **Etherophilic tasks require to preserve high frequencies**
- The Dirichelet **energy** $E(\mathbf{h})$ of model layers $\mathbf{h}$ can **measure this**
- Spatial view (embedding variability):
$$E(\mathbf{h}) = \sum_{v \in \mathcal{V}} \sum_{u \in \mathcal{N}_v} \|\mathbf{h}_v - \mathbf{h}_u\|^2$$

- **Spectral view**: *concise measure of the distribution on the graph frequency spectrum*
E.g. high frequency signal $\rightarrow$ **high energy**

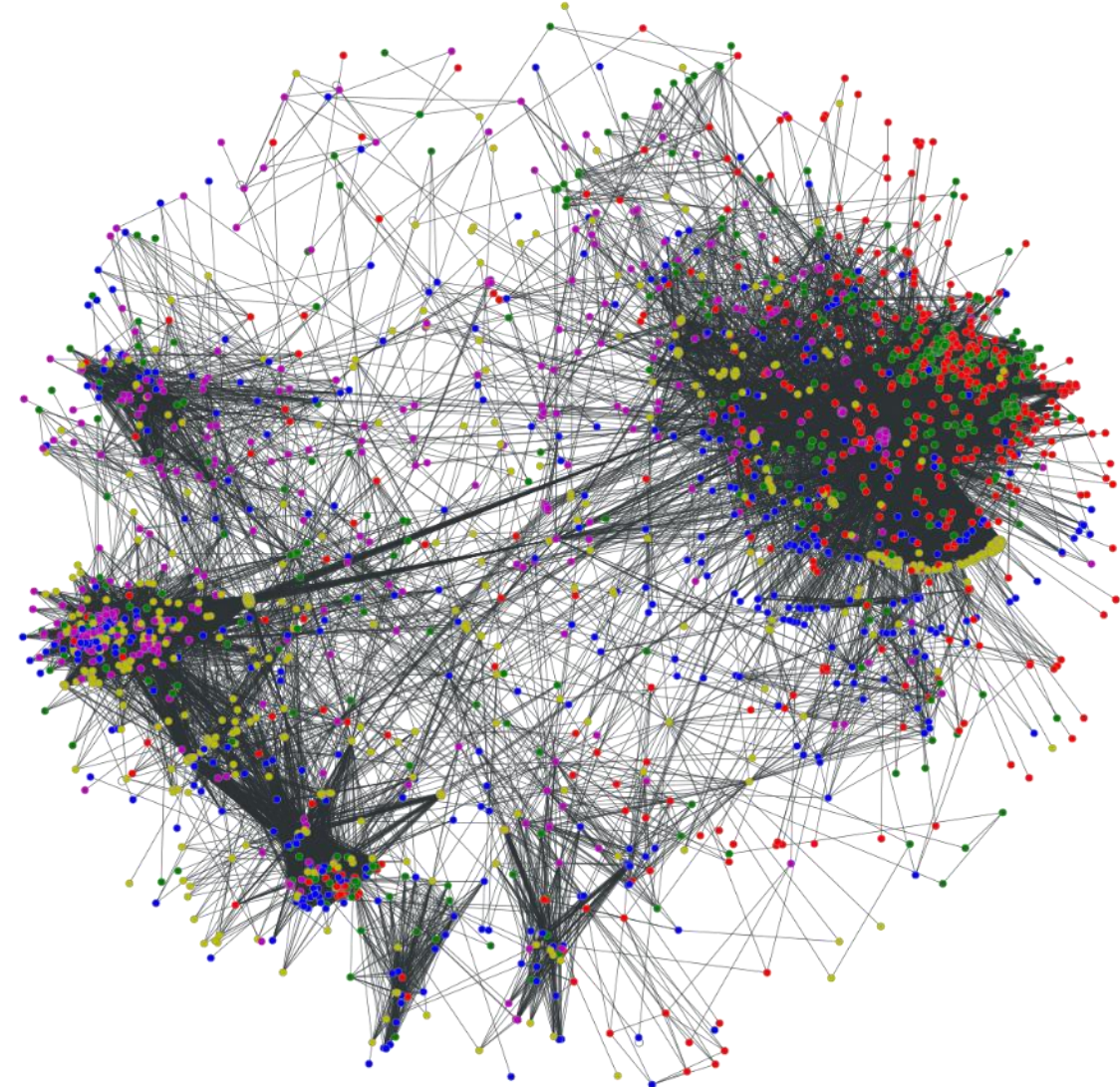


The Task: Chameleon (a «stress test») (Tech)

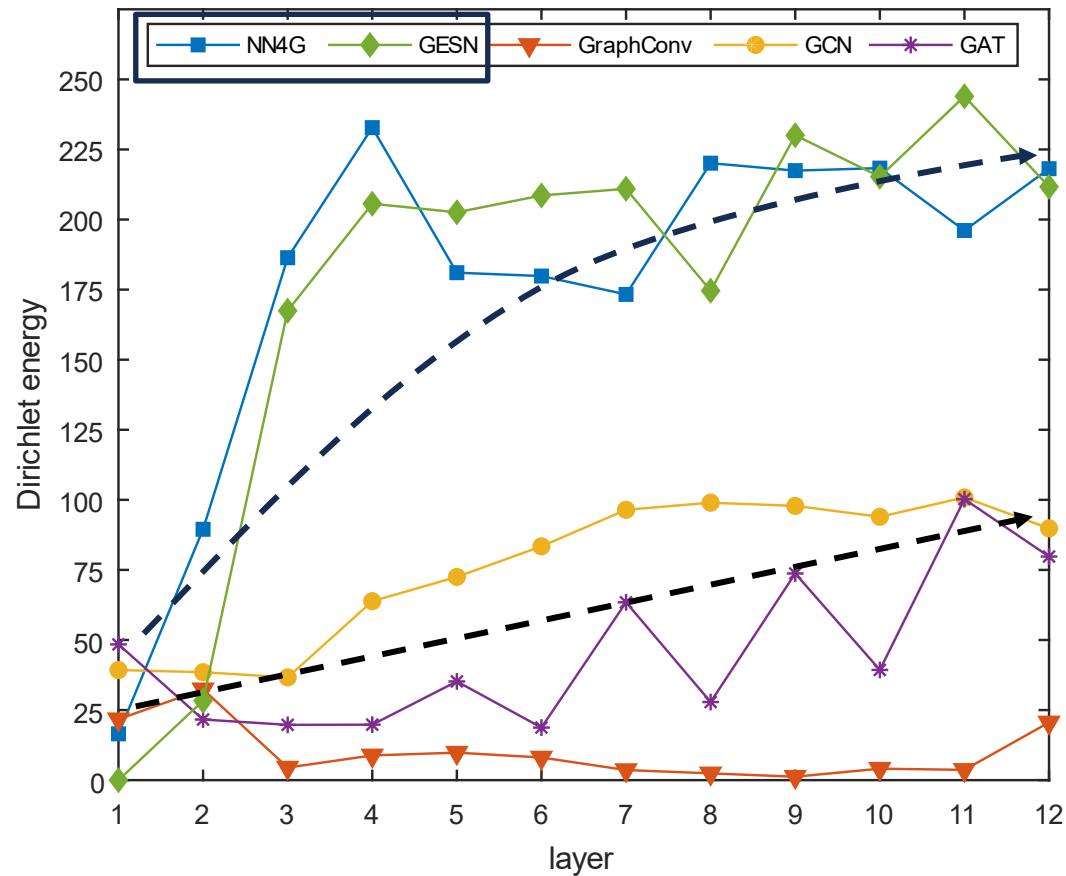
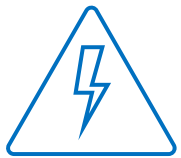


- Chameleon [Rozemberczki et al, 2019; Pei et al., 2020]
- Page-page network on a specific topics in Wikipedia
 - nodes represent web pages (bag of words)
 - edges are mutual links between pages
- 5 categories in term of the average monthly traffic of the web page
- Nodes: 2277, Edges: 31400
- Degree min/median/mean/max = 1/12/27.6/732
- Diameter: 11 (length of the "longest shortest path")
- Homophily, intra-class edges ratio [0-1]: 0.23

$$\frac{\text{intra-class edges}}{\text{total edges}}$$



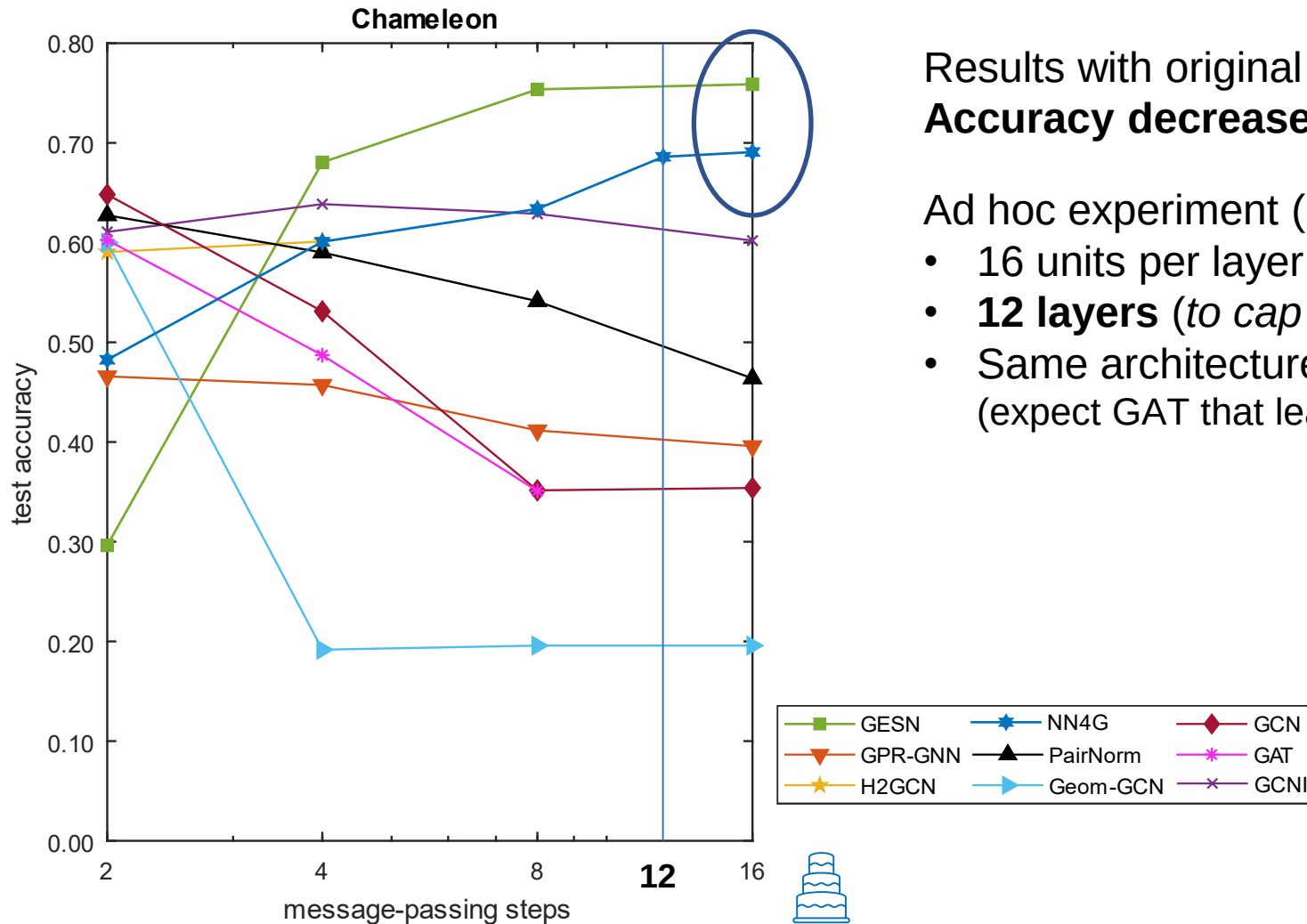
Spectral Analysis (Tech)



NN4G and GESN
better **preserve**
energy through layers
(MP)



Accuracy (Tech)



Results with original models [Yan, Y., et al. ICDM 2022]:
Accuracy decreases with MP, except for NN4G and GESN

Ad hoc experiment (Chameleon):

- 16 units per layer (except GESN)
- **12 layers** (to cap the graph diameter)
- Same architectures and units configurations (except GAT that learn edge weights via attention mech.)

GESN	73.7
NN4G	68.3
GCN	57.8
GAT	55.0
GraphConv	33.8



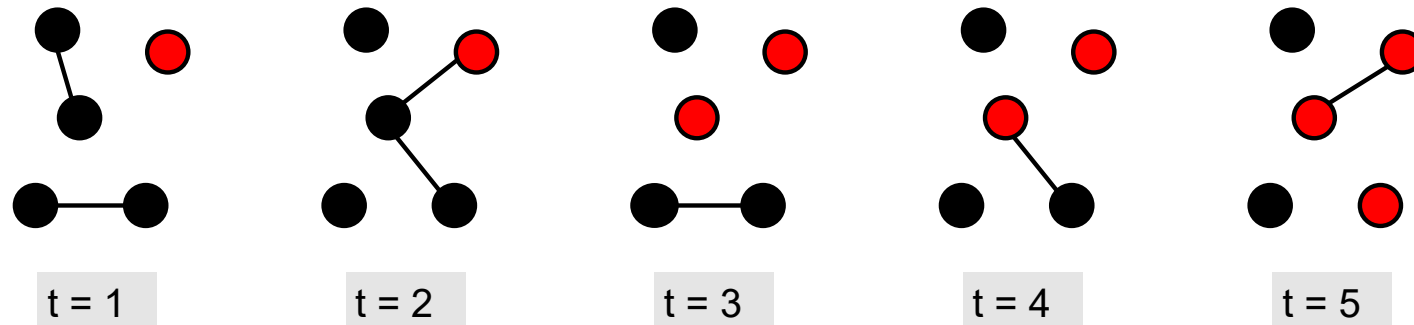
EXTENSIONS

On going research

Scope: Dynamical/temporal graphs



- Dynamic/Temporal Graphs represent relationships or node values that evolve through time
 - E.g. Infection-spreading models, content dissemination on social networks, signals on nodes...



A. Micheli, D. Tortorella. *Discrete-time Dynamic Graph Echo State Networks*. Neurocomputing (2022)

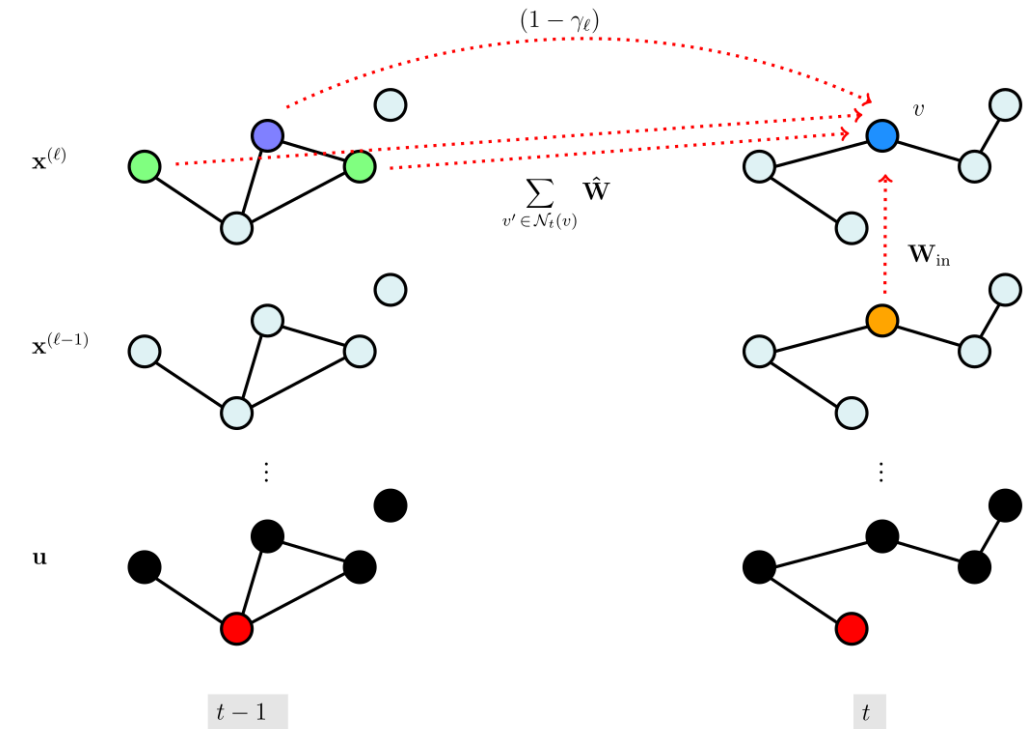
Dynamic Graph Echo State networks: Results



- Multi-layer reservoir defined by the node-wise state transition function, implementing a spatio-temporal graph convolution:

$$\mathbf{x}_t^{(\ell)}(v) = \gamma_\ell \tanh \left(\mathbf{W}_{\text{in}}^{(\ell)} \mathbf{x}_t^{(\ell-1)}(v) + \sum_{v' \in \mathcal{N}_t(v)} \widehat{\mathbf{W}}^{(\ell)} \mathbf{x}_{t-1}^{(\ell)}(v') \right) + (1 - \gamma_\ell) \mathbf{x}_{t-1}^{(\ell)}(v)$$

- **Stability condition:** $\|\widehat{\mathbf{W}}^{(\ell)}\| < \frac{1}{\|\mathbf{A}_t\|}$ for all layers
- More or as effective as (10) TGNs (fully-trained Temporal Graph neural Nets)
 - **100x** training speed-up w.r.t. TGNs
 - **10x** inference speed-up w.r.t. TGNs

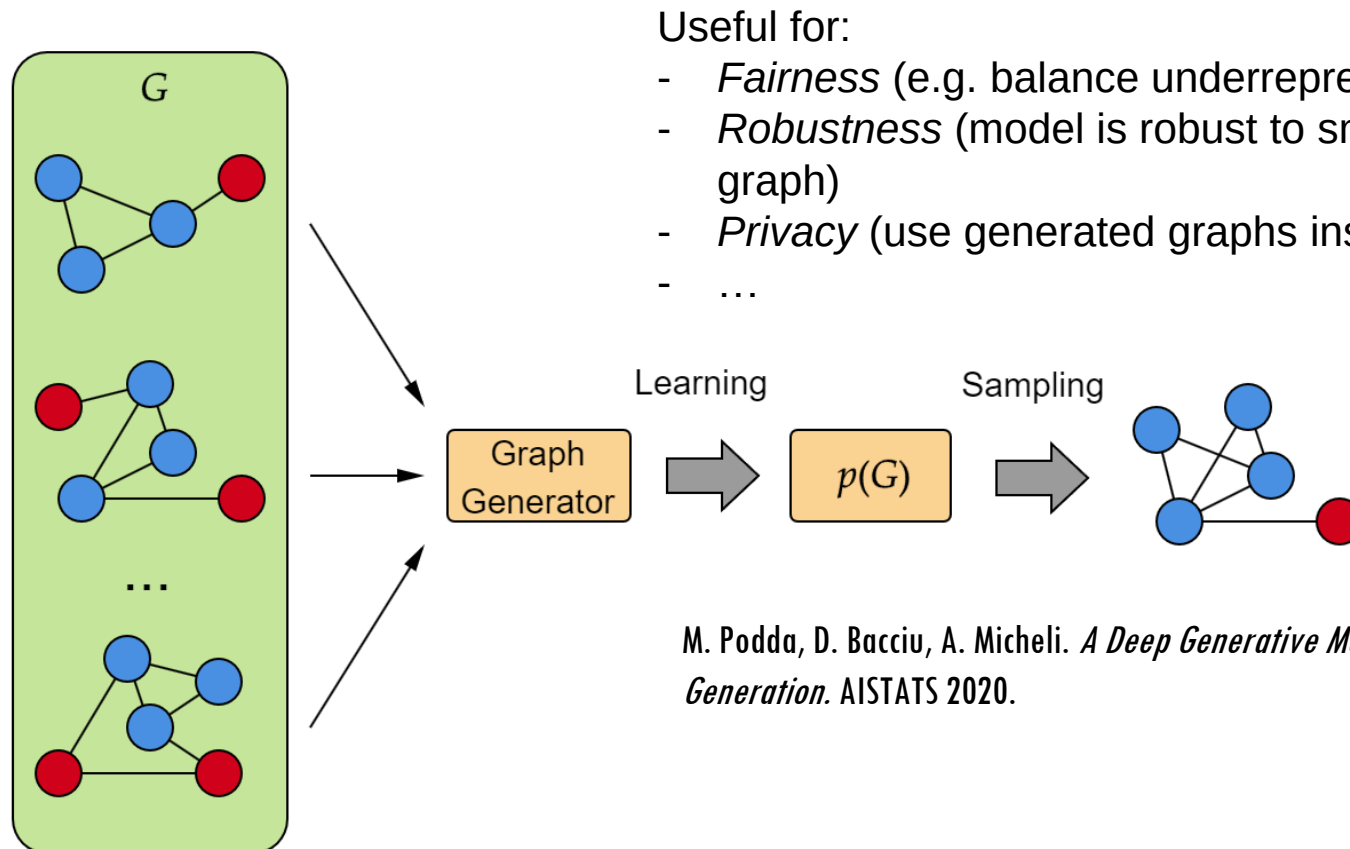


Micheli, Tortorella. *Discrete-time Dynamic Graph Echo State Networks*. Neurocomputing (2022).

Scope: Graphs generation



Goal: learn the probability distribution $p(G)$ underlying a set of graphs G to *sample novel instances*



Useful for:

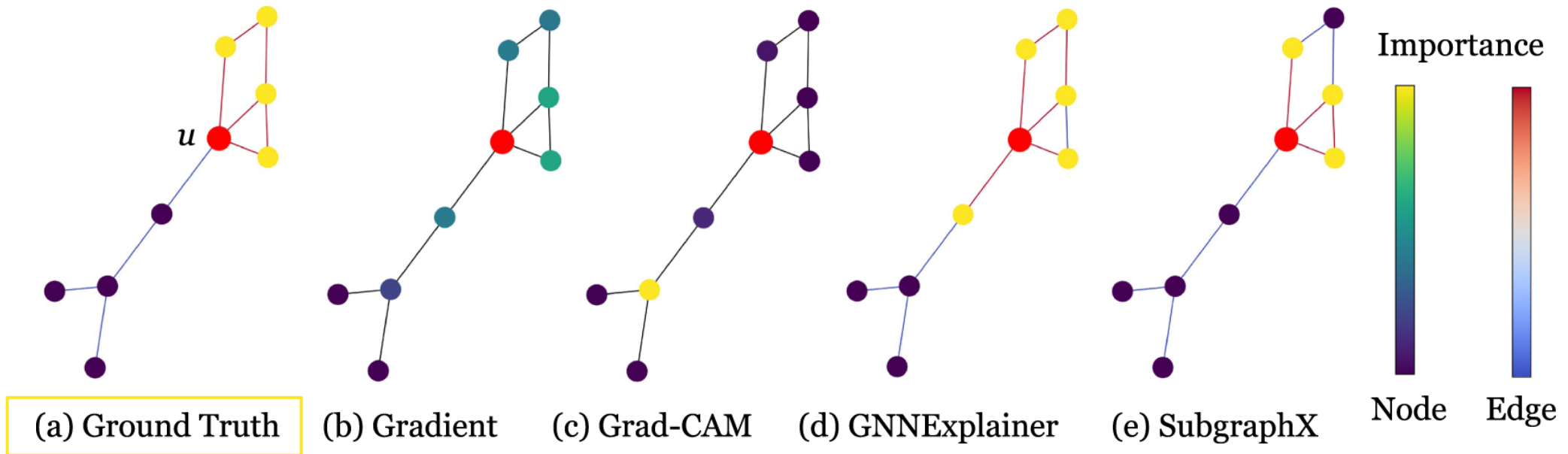
- *Fairness* (e.g. balance underrepresented groups)
- *Robustness* (model is robust to small variations of the input graph)
- *Privacy* (use generated graphs instead of real ones)
- ...

M. Podda, D. Bacciu, A. Micheli. *A Deep Generative Model for Fragment-Based Molecule Generation*. AISTATS 2020.

Explainability for graph neural networks



Goal: e.g. *explaining the prediction* of node u (or for a graph g)



Agarwal, C., Queen, O., Lakkaraju, H. *et al.* Evaluating explainability for graph neural networks. *Sci Data* **10**, 144 (2023)

Example on Molecules (I)



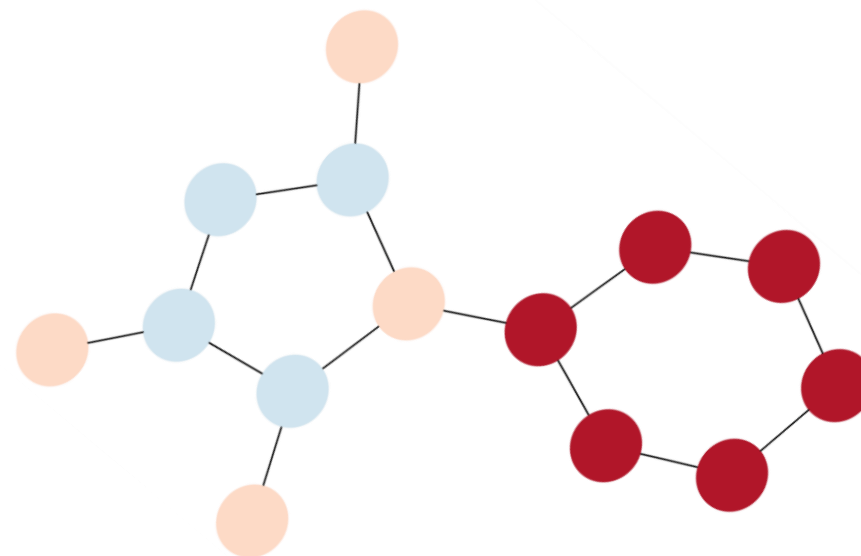
TASK – Classify the presence of benzene rings in molecules

Trained
Deep Graph
Network

"Benzene" dataset

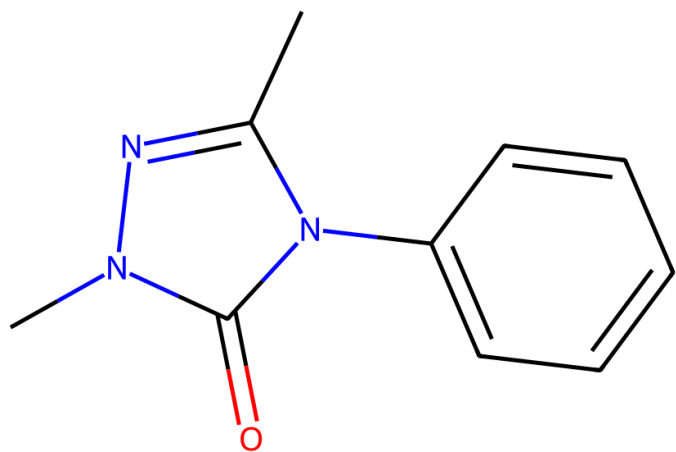
12 K from Zinc15. 50% positive.

XAI
Technique



LOW

HIGH



Applications on Molecules: Tox21 P53



NIH Tox21 Data Challenge 2014 (reduced and balanced, 1064/8634 samples)

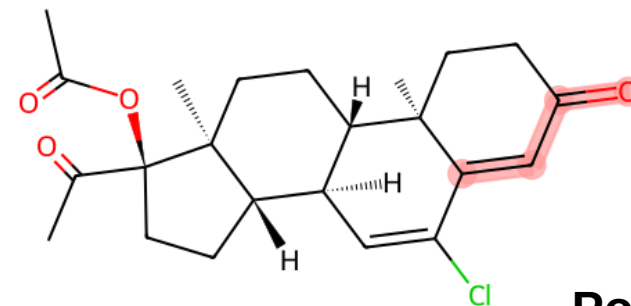
Predicting the activation of the **pathway P53** (0/1) directly from the molecular graph

XAI: Identify the (known) **fragment(s)** (structural patterns) that influences the activation

e.g.

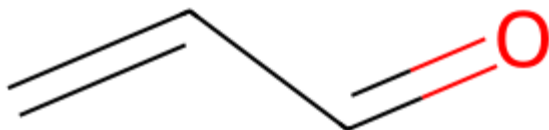


Michael acceptor



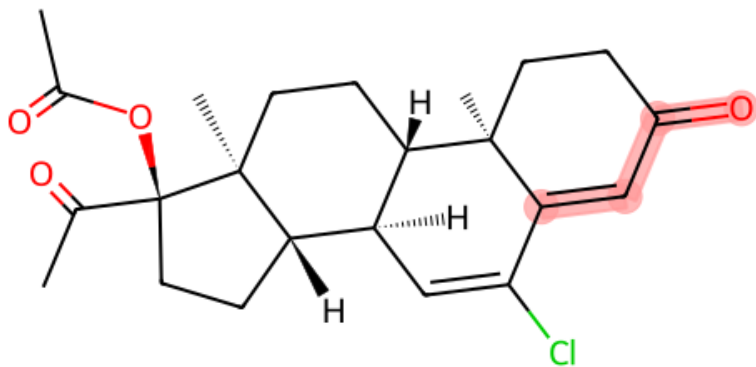
Positive

Applications on Tox21 P53 (I) Michael acceptor



Structural Pattern: **Michael acceptor**

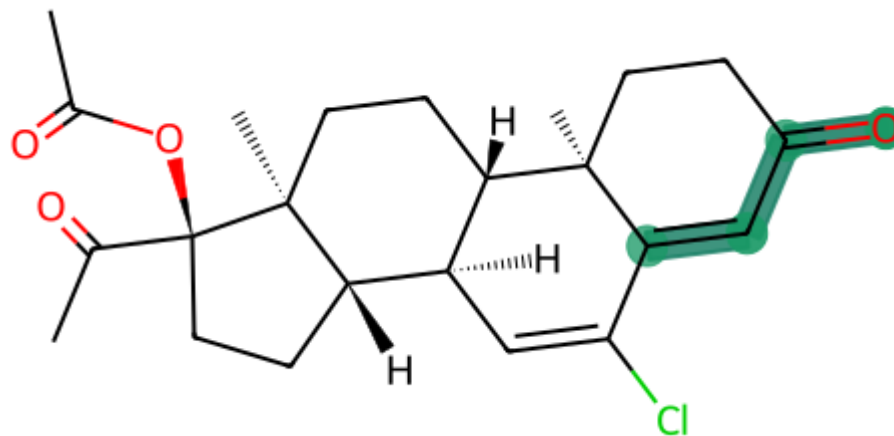
Smart: "[C]=[C]-[C](=[O])"



Name: chlormadinone acetate

Smile: [H][C@@]12CC[C@](OC(C)=O)(C(C)=O)[C@@]1(C)CC[C@@]3([H])[C@@]2([H])C=C(Cl)C4=CC(=O)CC[C@]34C

Result



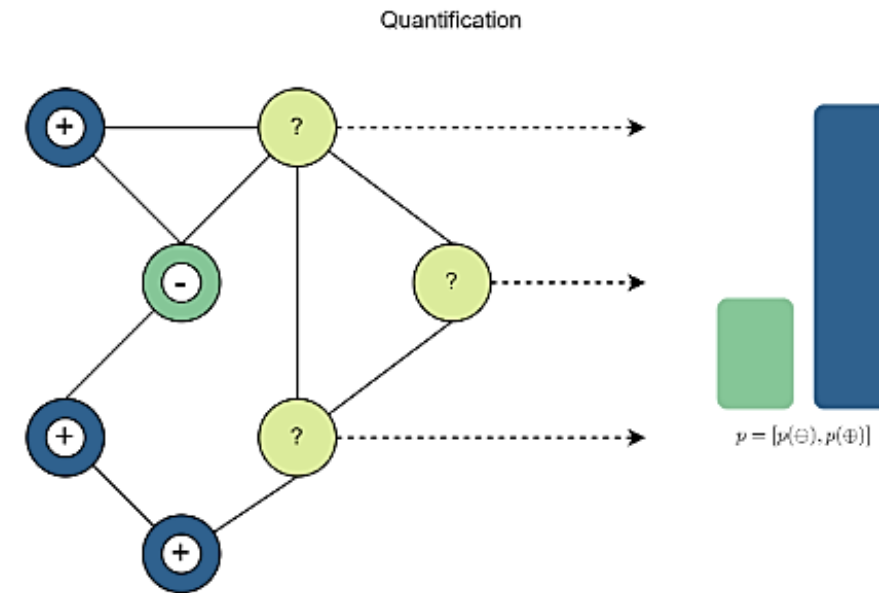
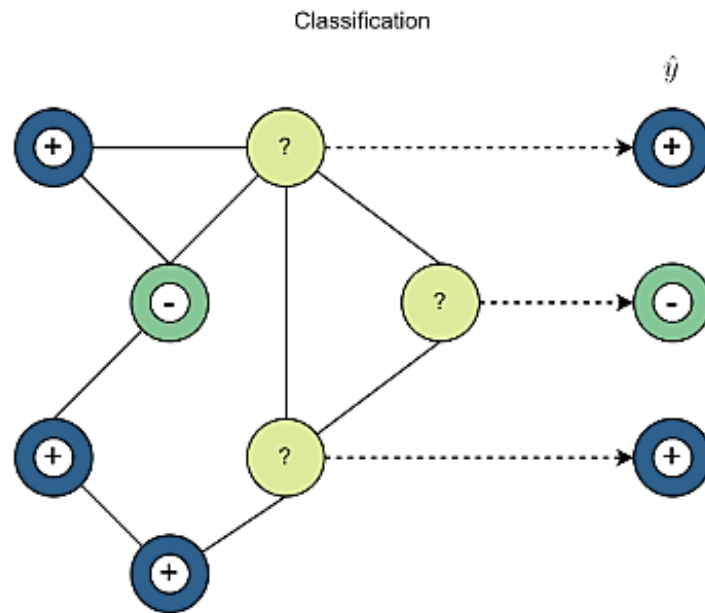
Top4 Nodes

Network Quantification



Goal: predict *the class prevalence of nodes* (distribution of the unlabeled nodes) instead of classify them individually

Examples: predict voting, market research problems, epidemiology problems, ...



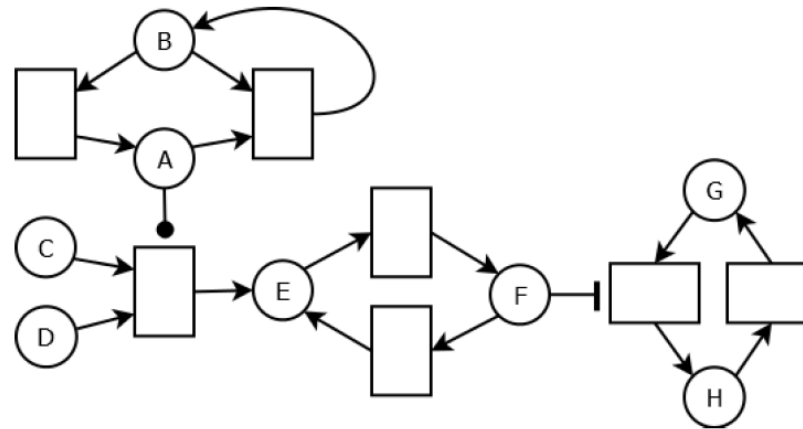
Applications: An example in Bioinformatics (I)



Biochemical Pathways (BPs) are sets of chemical reactions among biomolecules
They model **cell functionality**

Structure described by **graphs** (e.g. **Petri Nets**)

Goal: predict dynamical properties associated to the pathways (robustness, sensitivity, etc.)

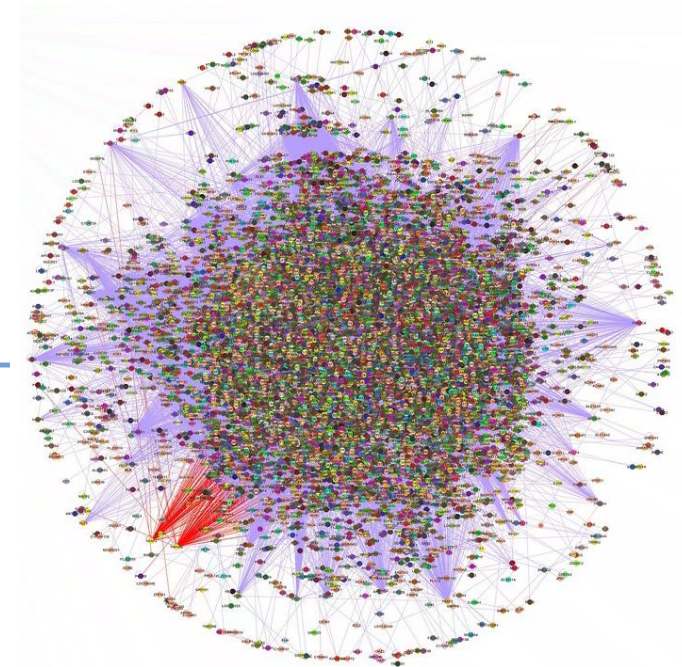
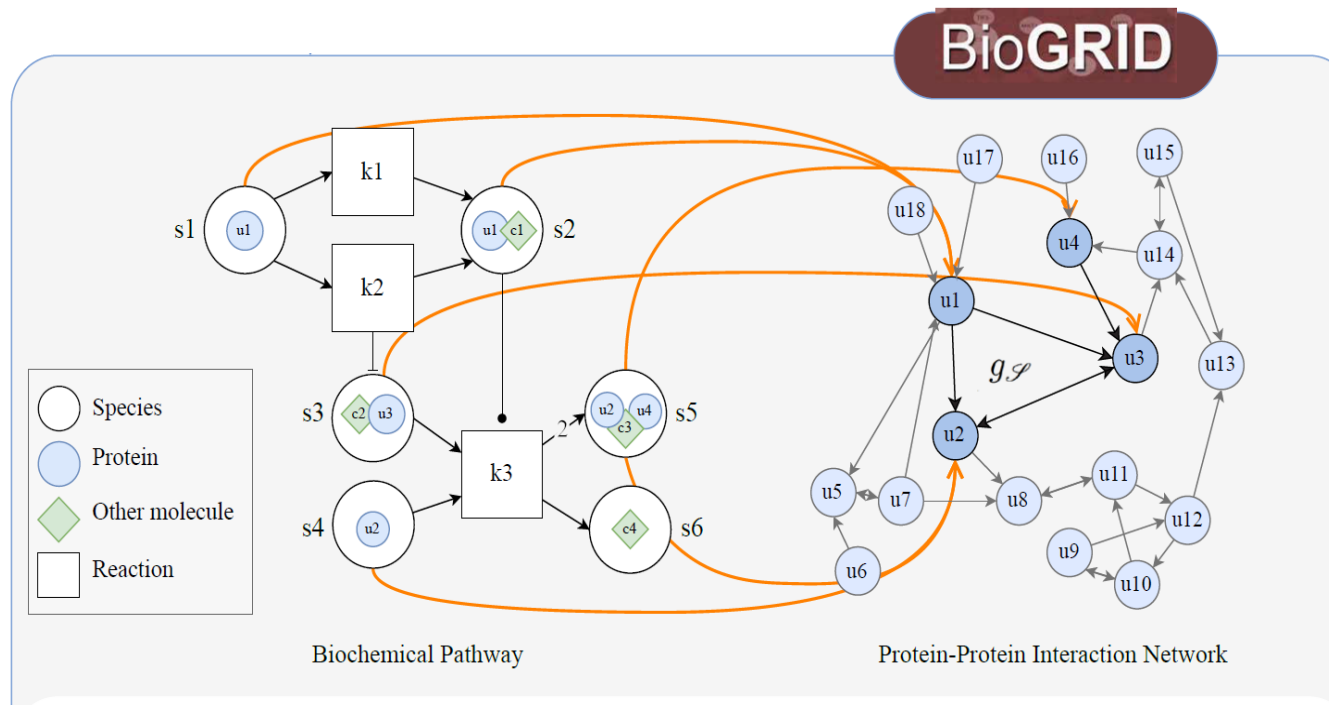


Fontanesi, M., Micheli, A., Milazzo, P., & Podda, M. (2023). Exploiting the structure of biochemical pathways to investigate dynamical properties with neural networks for graphs. *Bioinformatics*, 39(11)

Applications: An example in Bioinformatics (II)



Extension: make the same analysis on the more general case of Protein-Protein interaction Networks (PPIN)

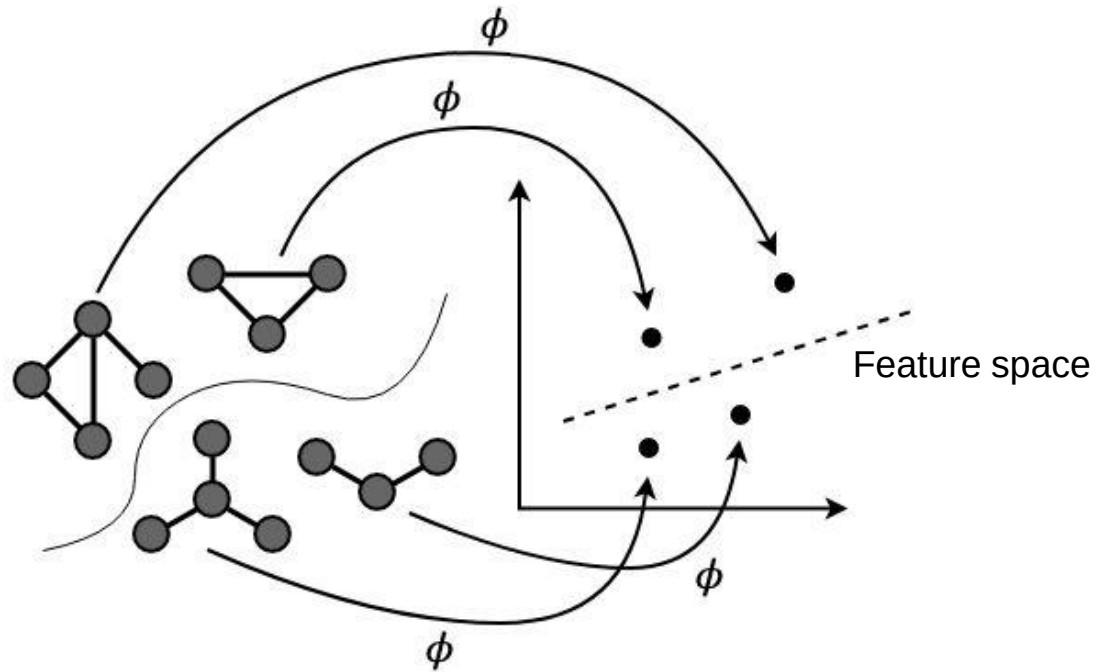


LEARNING IN SD

Other approaches

And Kernels?

- Large set of proposals (out of the scope)





Kernel methods for SD

Kernel modularity/ definition: easily can be extended to graphs of different types: sequences, trees, **graphs** (directed, undirected, cyclic/acyclic, ...)

Implicit embedding of data into Euclidean space by specifying an inner product

$$k(x, x') = \langle \phi(x), \phi(x') \rangle$$

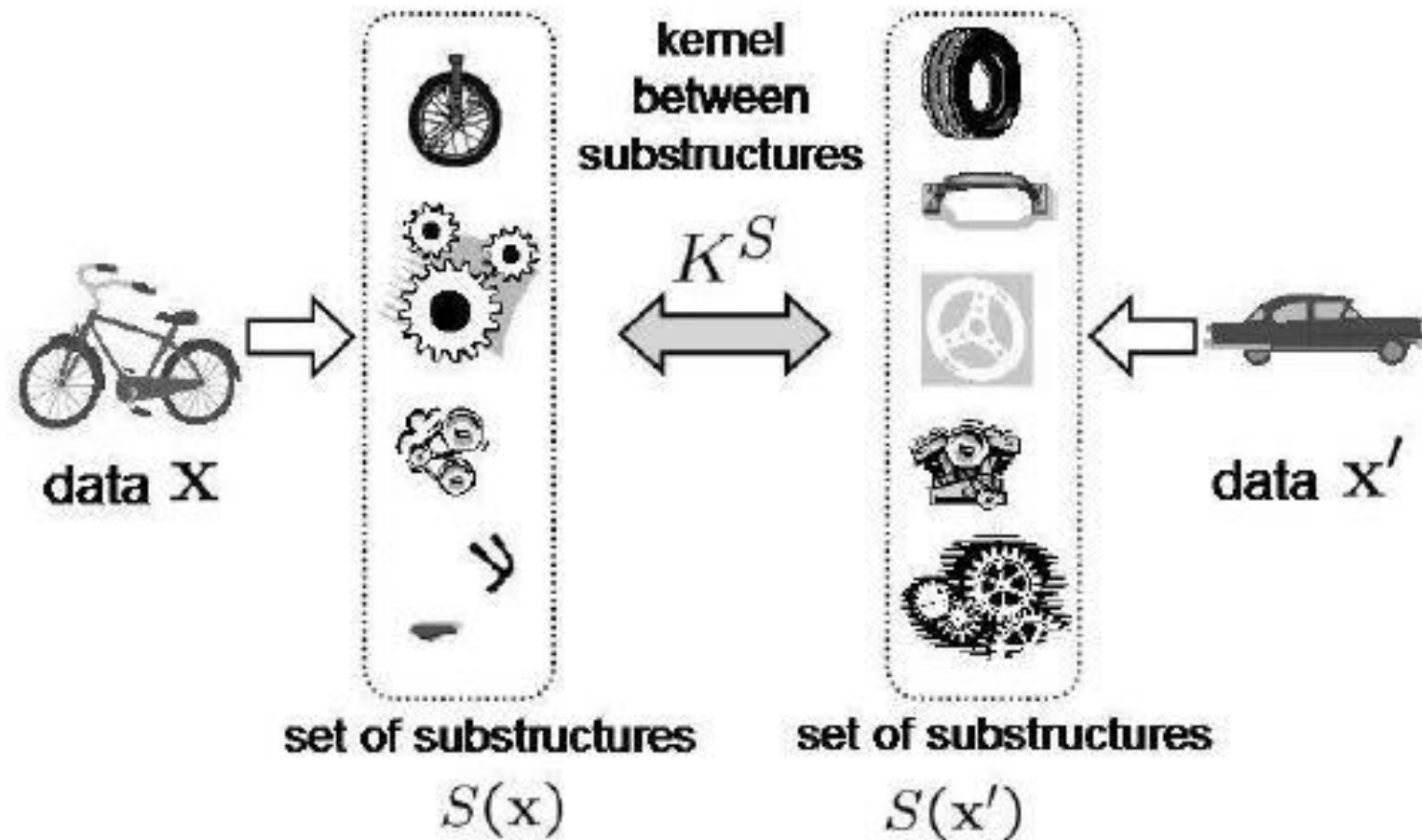
Data $\mathbf{X}$ can be of various types, including structured data

Define similarity between data

Example: Marginalized Kernel

- Similarity between 2 graphs based on the *common label paths* obtained by *random walks* in the graphs
- Kernel function defined as the inner product of the count vectors, averaged over all possible label paths
- Computational complexity of the kernel function (typically) scales quadratically with the graph size

Convolution Kernels



Kernels for SD: discussion

- Efficiency: the cost is critical (due to sub-structure decomposition)
- **Adaptivity warning:** Kernel for structure: *knowledge-driven methods*
- Similarity measure fixed prior to learning
 - i.e. no adaptivity/learning in the encoding into the feature space
 - **Pro:** we can exploit prior knowledge by the Kernel (to engineering a specific problem)
 - **Cons:** For SD no universality is known:
 - The choice of the kernel function is critical
 - E.g. Computing any complete graph kernel is at least as hard as '**Graph Isomorphism**'
- Adaptive kernels studies...

On going research

- Research for extension of **DGN** to **continual learning, automatic model design** within the **trustworthy** concepts (*explainability*, robustness, fairness, privacy) is open....
(graphs are discrete structures!)
- PNRR PE00000013 "FAIR - Future Artificial Intelligence Research" - Spoke 1 "**Human-centered AI**", funded by the European Commission under the *NextGeneration EU programme*. 2023, 3 years
- WP on *Human centered Lifelong Learning for Complex Data*



On going research: next project



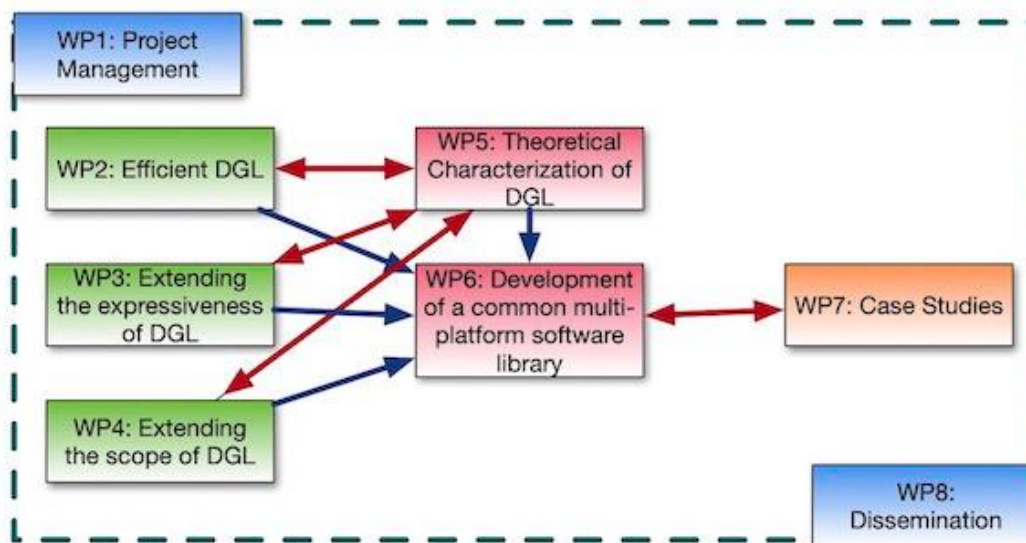
- PRIN (national project), 2Y, starting Feb. 2025
- **DEEP-GRAPH: Design and Theory of Deep Graph Learning**
 - University of Padova (Prof. A. Sperduti)
 - University of Pisa (Prof. A. Micheli)
 - University of Siena (Prof. F. Scarselli)



Finanziato
dall'Unione europea
NextGenerationEU



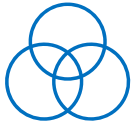
Ministero dell'Università e della Ricerca



Conclusions (I): Research



- It is possible to do DL for complex data **efficiently** (without fully E2E training)
 - Models as GESN and NN4G allow to efficiently build DGN with high accuracy,
 - at the same time enabling the **investigation** of the MP bias
- Interesting **interplay between depth** of the models and **complex data representation**:
 - The **depth** in DGN is needed
 - Depth can lead to issues, but **the fault is not of the MP, of the number of layers, of the graph topology**
 - **Open issue**: to better discriminate the reasons for each causes (over*, training issues, etc.)
 - E.g. we have at least shown that **going beyond E2E** can be an advantage for efficiency and to deal with over* phenomena



General conclusions



- **ML for SD/graphs** is a **concrete reality** (it can provide new solutions for complex domains where the *managing of relationships* is relevant to achieve suitable modeling of the solutions), while still under development
- If you have data with relationships (serial, hierarchical, networks), any domain where you find structured data, **do not use a flat representation** ... we can work together on them!
- A great opportunity to **treat uniformly different kind of data**

General conclusions



Many open issues &
extensions



Many applicative fields

We are open to collaborations!
Your Thesis and more  *“tie a knot in your
handkerchief”*

CIML group @ UNIPi

Contact:

Alessio Micheli

micheli@di.unipi.it

Thank You



Dipartimento di Informatica
Università di Pisa - Italy



**Computational Intelligence
& Machine Learning Group**

Site: ciml.di.unipi.it